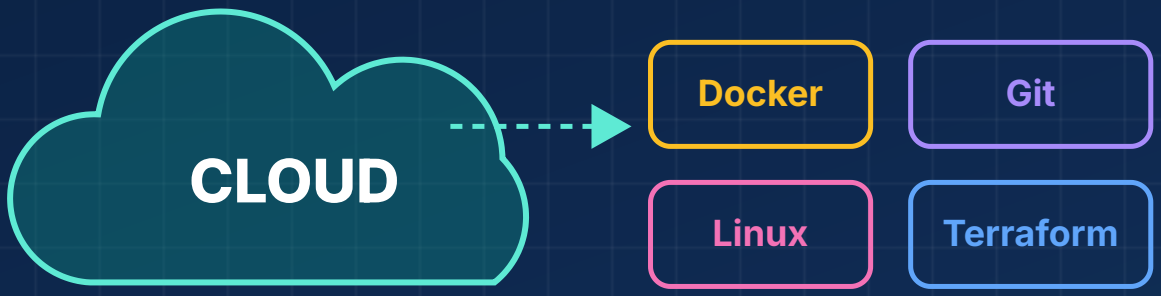
PRACTICAL STARTER GUIDE

Cloud & DevOps Engineering

A Practical Starter Guide

Understand modern software infrastructure.





01 Cloud computing fundamentals

02 AWS, Azure & Google Cloud

03 Linux & Git fundamentals

04 Docker & containers

05 CI/CD fundamentals

06 Deployment architecture & mini-projects

PrepPath E-books Store

100 pages · diagrams · hands-on labs

Contents

Chapter	What is inside	Page
1. Cloud Computing Fundamentals	Cloud models, regions, networking, storage, security, costs	4
2. AWS, Azure & Google Cloud	Service maps, AWS / Azure / GCP walk-throughs, choosing a cloud	20
3. Linux & Git Fundamentals	Shell, permissions, scripting, Git, branches, GitHub workflow	36
4. Docker & Containers	Images, Dockerfiles, volumes, networks, Compose, Kubernetes intro	56
5. CI/CD Fundamentals	Pipelines, GitHub Actions, deployments, IaC, monitoring	72
6. Architecture & Mini-Projects	Architecture patterns, 4 mini-projects, career roadmap, glossary	86

How the book is organised

Each chapter starts with a **big idea**, builds up with diagrams and short examples, and ends with a **hands-on lab** you can finish in one sitting. Read in order the first time; later, jump to the chapter you need.

Icon / box	Meaning
Key idea	The one concept you must remember
Try it	Run this yourself, right now
Pro tip	Habit used by working engineers
Watch out	A common mistake or a risky default
Dark block	Commands or files; output beneath was really executed

> **ABOUT THE EXAMPLES**
> Commands for AWS, Azure, Google Cloud and Terraform need your own cloud account, so they are shown as **illustrative** (no output). Linux, Git, Python, YAML and Dockerfile examples were checked before printing.

Who this guide is for

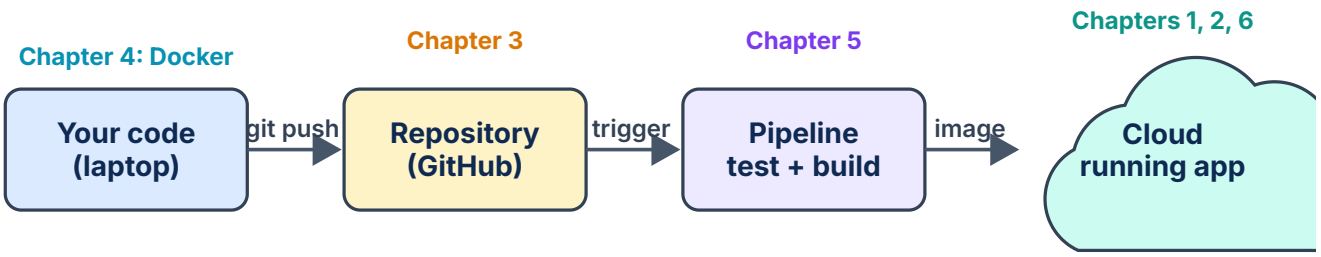
- Students and freshers who want a first job in cloud, DevOps or backend roles
- Developers who deploy code but do not yet understand what happens after `git push`

Welcome & Lab Setup

Modern software is not just code. It is code **plus** the machines, networks, pipelines and habits that put it in front of users. This guide walks that whole road, in the order you will meet it:



The journey in one picture



From a commit on your laptop to a running service in the cloud

Set up your lab (about 30 minutes)

1. **A Linux shell.** Linux or macOS terminal, or Windows with WSL 2 (Ubuntu).
2. **Git** and a free **GitHub** account. Check with `git --version`.
3. **Docker Desktop** (or Docker Engine on Linux). Check with `docker --version`.
4. **Python 3.10+** and **VS Code** (any editor works). Check with `python3 --version`.
5. One cloud free-tier account (AWS, Azure **or** Google Cloud). You do not need all three.

DO THIS FIRST

The moment you create a cloud account, set a **budget alert** and turn on **MFA** for the root user. Do this before launching anything. Chapter 1 explains why.

PRO TIP

Type every command yourself instead of copy-pasting. Typing builds muscle memory, and muscle memory is what interviews and incidents test.

What Is Cloud Computing?

01

CHAPTER 1

Cloud Computing Fundamentals

Rent computing power the way you rent electricity

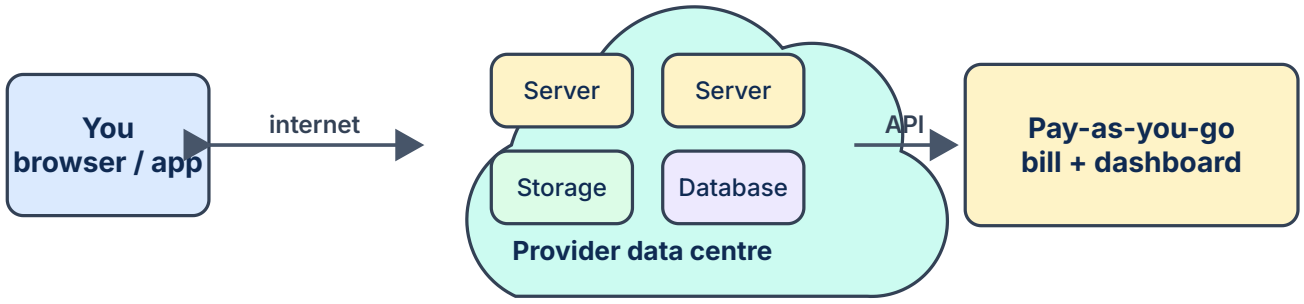
WHAT YOU WILL LEARN

- What the cloud really is and why companies adopt it
- IaaS, PaaS, SaaS and the deployment models
- Regions, scaling, availability and shared responsibility
- Networking, storage and serverless basics

Cloud computing is renting computing resources (servers, storage, databases, networks, software) over the internet, paying only for what you use, instead of buying and running your own hardware.

KEY IDEA

The US standards body NIST defines five essential characteristics: **on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service**. If a service has these, it is cloud.



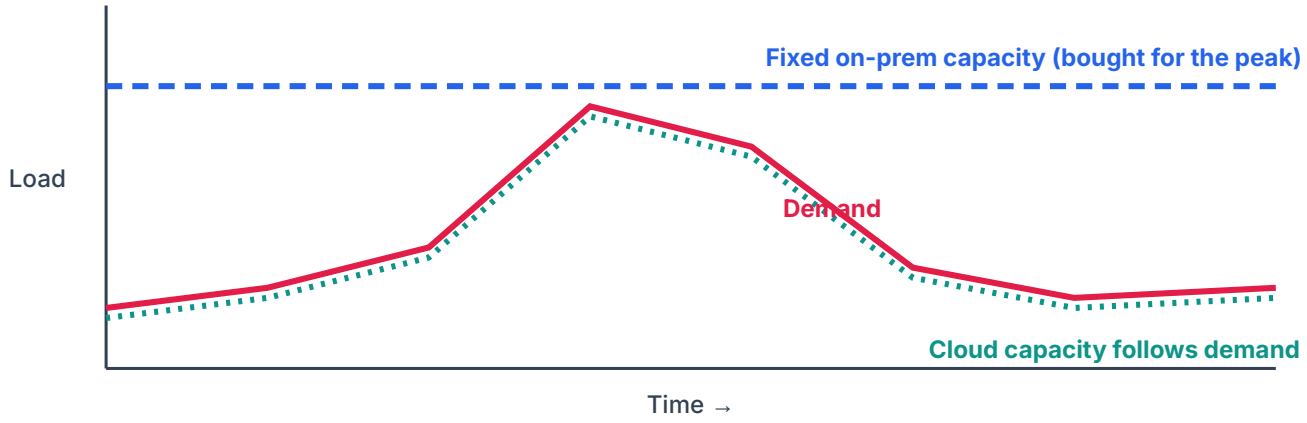
You rent slices of the provider's huge shared data centres through the internet

Why it took over

Benefit	What it means for you
Speed	A server in minutes, not weeks of purchase orders
Elasticity	Add capacity for a sale day, remove it afterwards
Global reach	Deploy near users in many countries
Managed services	Provider patches databases and hardware for you

Traditional IT vs the Cloud

Before the cloud, a start-up had to estimate its peak traffic, **buy servers for that peak**, wait for delivery, rack them and hire people to keep them alive. If traffic stayed low the hardware sat idle; if it spiked the site fell over.



Fixed hardware is wasted most of the time; cloud capacity can follow the demand curve

	On-premises	Public cloud
Upfront spend	Large (capital expense)	Near zero (operating expense)
Time to get a server	Weeks to months	Minutes
Capacity	Fixed, guess the peak	Grow and shrink on demand
Maintenance	Your team: hardware, power, cooling	Provider handles the facility and hardware
Control	Total	High, but within provider limits
Best for	Strict data-location rules, steady loads, legacy systems	Variable traffic, fast growth, global apps

KEY IDEA

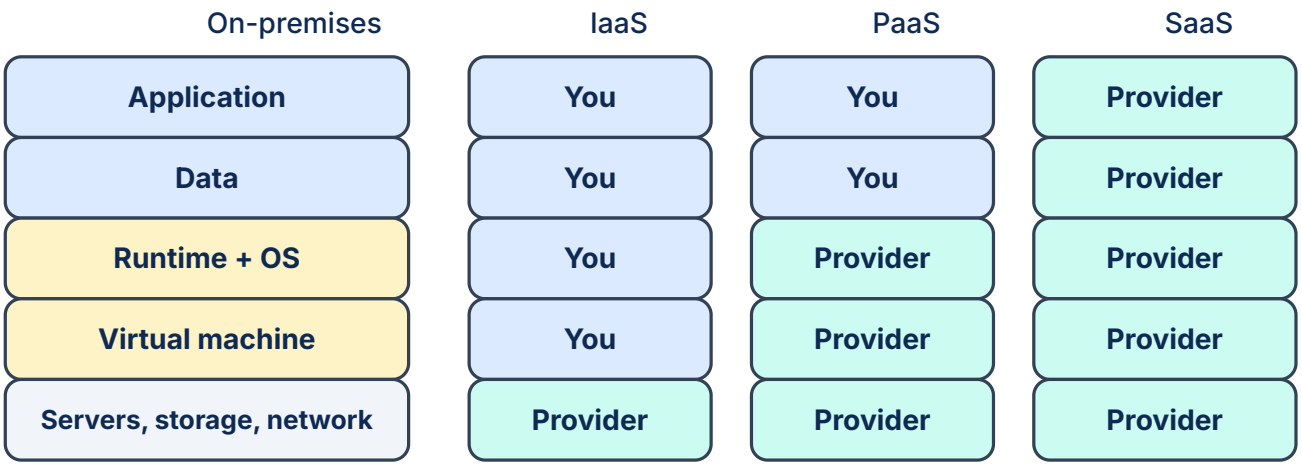
CapEx (capital expense) is money spent up front on assets. **OpEx** (operating expense) is a recurring bill for usage. The cloud turns CapEx into OpEx. Finance teams care a lot about this switch.

HONEST NOTE

The cloud is not always cheaper. A steady, predictable, heavy workload can cost less on owned hardware. Always compare using **total cost of ownership**: hardware, power, space, staff time and downtime risk.

IaaS, PaaS and SaaS

Cloud services come in layers. The higher the layer you rent, the less you manage and the less you can customise.



Blue = you manage Teal = the provider manages

Who manages which layer

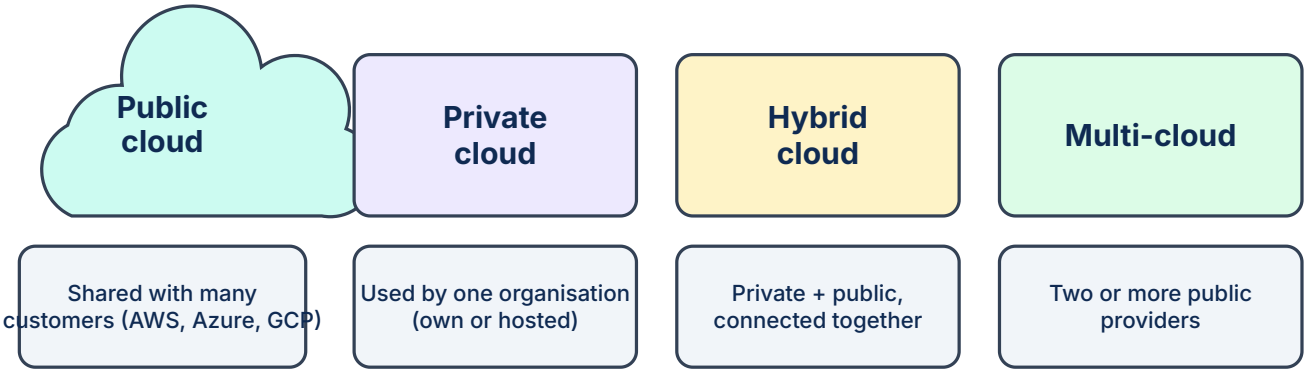
Model	You get	Typical examples	Good for
IaaS	Virtual machines, disks, networks	AWS EC2, Azure Virtual Machines, Google Compute Engine	Full control, lift-and-shift of old apps
PaaS	A platform to run your code	AWS Elastic Beanstalk, Azure App Service, Google App Engine	Shipping web apps without server chores
SaaS	A finished application	Gmail, Microsoft 365, Slack, Salesforce	Using software, not building it
FaaS	Run a function per event	AWS Lambda, Azure Functions, Cloud Run functions	Event-driven, spiky workloads

MEMORY TRICK

Pizza analogy: **on-prem** = make it at home, **IaaS** = buy a ready base and add toppings, **PaaS** = takeaway, you just eat, **SaaS** = dine at a restaurant.

Cloud Deployment Models

Where does the cloud physically live, and who may use it? There are four common answers.



Four deployment models

Model	Why choose it	Trade-off
Public	Fastest start, no hardware, huge service catalogue	Shared infrastructure, provider rules apply
Private	Tight control and compliance	You build and run it, higher effort
Hybrid	Keep sensitive data on-site, burst to cloud for peaks	Complex networking and security
Multi-cloud	Avoid lock-in, use the best tool per provider	Skills and tooling must cover each cloud

Example: a hospital app

Patient records stay in a **private** data centre because of regulation. The appointment website runs on a **public** cloud so it scales during flu season. The two talk over an encrypted link. That combination is **hybrid cloud**.

PRO TIP

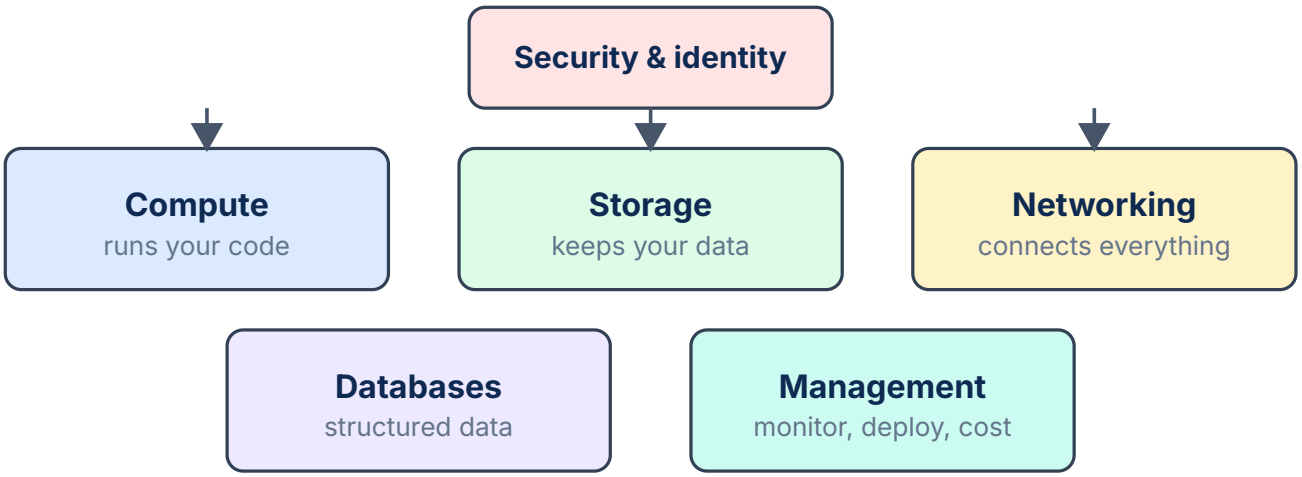
Most beginners should learn **one public cloud deeply** first. The concepts transfer to the others, as Chapter 2 shows.

WATCH OUT

Multi-cloud sounds safer than it is. Every extra provider doubles the things your team must secure, monitor and pay for. Do it for a clear reason, not as a default.

Building Blocks of the Cloud

Every cloud, however many hundred services it advertises, is made from five basic building blocks. Learn these and the product catalogues stop looking scary.



The five building blocks, wrapped in security

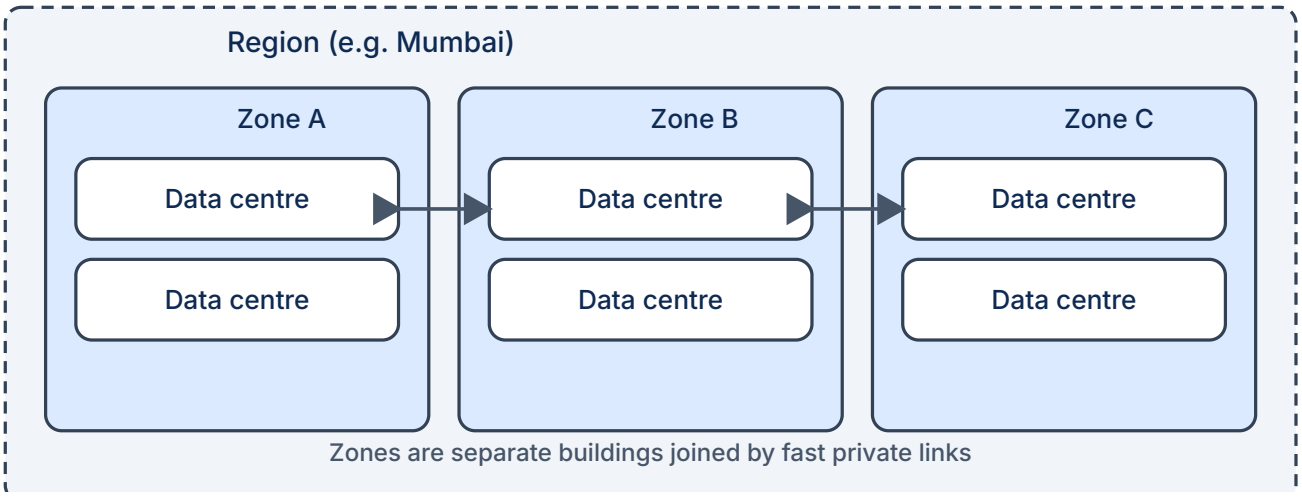
Block	Question it answers	Typical services (AWS / Azure / GCP)
Compute	Where does my code run?	EC2 / Virtual Machines / Compute Engine
Storage	Where do files live?	S3 / Blob Storage / Cloud Storage
Networking	How do parts talk, and who is allowed?	VPC / Virtual Network / VPC
Databases	Where is structured data kept?	RDS / Azure SQL / Cloud SQL
Security	Who can do what?	IAM / Entra ID + RBAC / Cloud IAM
Management	How do I watch and automate it?	CloudWatch / Azure Monitor / Cloud Monitoring

KEY IDEA

When you meet a new cloud service, ask: *is this compute, storage, network, database, security or management?* That single habit makes any catalogue readable.

Regions, Zones and Edge

Providers do not run one giant computer. They run **many data centres** grouped into a hierarchy.



One region contains several isolated zones

Term	Meaning
Region	A geographic area with several data centre groups (Mumbai, Frankfurt, Virginia)
Availability Zone (AZ)	One or more separate data centres in a region with independent power and cooling
Edge location / PoP	Small sites near users that cache content for speed (CDN)

How to choose a region

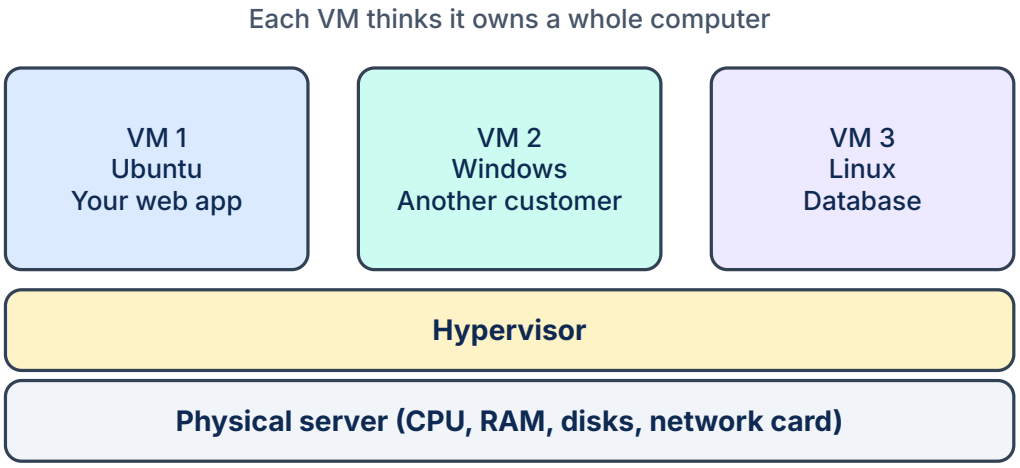
1. **Latency**: pick the region closest to most users.
2. **Data rules**: some laws require data to stay inside a country.
3. **Service availability**: new services appear in big regions first.
4. **Price**: the same service can cost differently per region.

PRO TIP

Putting copies of your app in **two or more zones** protects you if one building loses power. This is the cheapest big reliability win in the cloud.

Virtualization and Compute

A **hypervisor** is software that slices one physical server into many isolated **virtual machines (VMs)**, each with its own operating system. This is how a provider can rent you two CPUs and 4 GB of RAM from a machine that has far more.



Virtualization: many isolated machines on one server

Compute options, smallest to biggest abstraction

Option	What you manage	Starts in
Bare metal	Everything, including the OS on dedicated hardware	Minutes to hours
Virtual machine	OS, runtime, app	Minutes
Container	App and its libraries (shared OS kernel)	Seconds
Serverless function	Only the function code	Milliseconds to seconds

Reading an instance type name

Names such as `t3.medium` (AWS), `Standard_B2s` (Azure) or `e2-medium` (Google) encode a **family** (purpose) and a **size** (vCPU + memory).

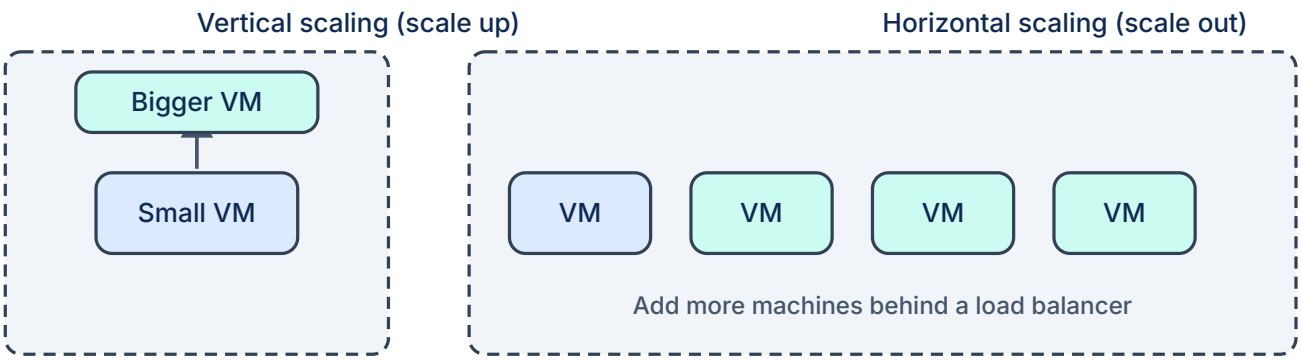
Family idea	Choose it when
General purpose	Web servers, small databases, dev/test
Compute optimised	CPU-heavy work such as video encoding
Memory optimised	In-memory caches, big databases

WATCH OUT

Start **small**: a forgotten large VM quietly burns money every hour.

Scalability and Elasticity

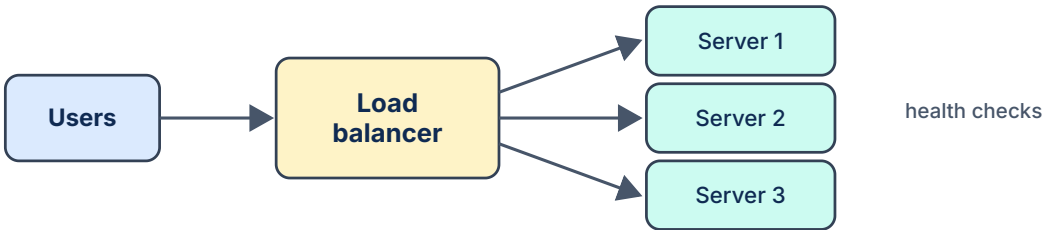
Scalability is the ability to handle more load. **Elasticity** is doing it *automatically*, both up and back down.



Bigger machine vs more machines

	Vertical	Horizontal
How	Add CPU/RAM to one machine	Add more machines
Limit	Biggest machine available	Practically unlimited
Downtime	Often a restart	None, if designed well

The load balancer



A load balancer spreads requests and skips unhealthy servers

Auto scaling in one rule

```
Policy
If average CPU > 70% for 5 minutes  →  add 1 server
If average CPU < 30% for 10 minutes →  remove 1 server
Always keep between 2 and 10 servers
```

KEY IDEA

Horizontal scaling works best when servers are **stateless**: they keep no unique data locally. Session data, uploads and orders belong in a shared database or storage service.

High Availability and Disaster Recovery

Availability is the percentage of time a service works. It is usually quoted in *nines*. Run the maths yourself:

availability.py

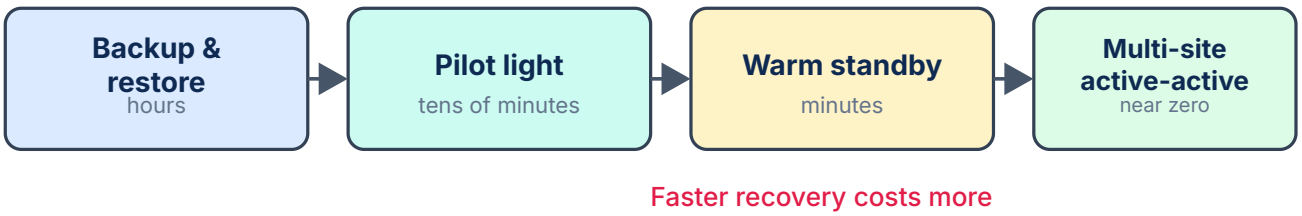
```
def downtime_per_year(availability_percent):
    minutes = 365 * 24 * 60
    return minutes * (1 - availability_percent / 100)

for a in (99.0, 99.9, 99.99, 99.999):
    print(f"{a}% → {downtime_per_year(a):.0f} minutes of downtime a year")
```

```
99.0% → 5256 minutes of downtime a year
99.9% → 526 minutes of downtime a year
99.99% → 53 minutes of downtime a year
99.999% → 5 minutes of downtime a year
```

Term	Meaning	Example
High availability	Design so a failure does not stop the service	Servers in two zones
Fault tolerance	Keep working with zero interruption while a part fails	Mirrored disks
Disaster recovery (DR)	Get back after a big failure	Restore in another region
RTO	Recovery Time Objective: how long you can be down	Back in 1 hour
RPO	Recovery Point Objective: how much data you can lose	Lose at most 5 minutes

Four DR strategies, cheapest to fastest



More money buys a shorter RTO and RPO

WATCH OUT

A backup you have never **restored** is only a hope. Schedule a restore test and write down how long it took.

Shared Responsibility and IAM

Security in the cloud is a **partnership**. The provider secures the cloud itself; you secure what you put in it.

Provider: security OF the clo	You: security IN the cloud
Buildings and power	Who has access (IAM)
Hardware and hypervisor	Your data and encryption choices
Global network	OS patches on your VMs
Managed-service internals	Network rules and app code

The line moves depending on IaaS, PaaS or SaaS

IAM: identity and access management

- **User**: a person or app with credentials
- **Group**: a set of users sharing permissions
- **Role**: permissions that a user or service can temporarily assume
- **Policy**: a document listing allowed or denied actions on resources

Example AWS-style policy: read-only access to one bucket

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::my-notes-bucket/*"
    }
  ]
}
```

KEY IDEA

Least privilege: grant only the permissions needed, only for as long as needed. Never use the root account for daily work, and always enable **MFA**.

WATCH OUT

Never put access keys in source code or a public repository. Bots scan GitHub for them within minutes. Use roles and secret managers instead (Chapter 5).

Cloud Pricing and Cost Control

The cloud bills by usage. That is freedom if you understand it and a nasty surprise if you do not.

Pricing model	How it works	Best for
On-demand	Pay per second or hour, no commitment	Unpredictable or short workloads
Reserved / committed	Promise 1 to 3 years of use for a discount	Steady, always-on workloads
Spot / preemptible	Spare capacity at a big discount; can be taken back with short notice	Batch jobs that tolerate interruption

What actually appears on the bill

- **Compute**: hours that VMs are running (even if idle)
- **Storage**: GB stored per month, plus requests
- **Data transfer**: traffic *out* of the cloud usually costs, traffic in is usually free
- **Managed services**: databases, load balancers, NAT gateways bill per hour

cost_estimate.py

```
def monthly_vm_cost(hourly_rate, hours=730):
    return round(hourly_rate * hours, 2)

rate = 0.02          # example rate per hour, not a real quote
print("Running 24/7 :", monthly_vm_cost(rate))
print("Office hours :", monthly_vm_cost(rate, hours=9 * 22))
```

```
Running 24/7 : 14.6
Office hours : 3.96
```

Seven cost habits

- ☐ Set a **budget alert** on day one
- ☐ **Tag** every resource with owner and project
- ☐ Delete unattached disks, snapshots and idle load balancers
- ☐ Review the bill weekly while learning

WATCH OUT

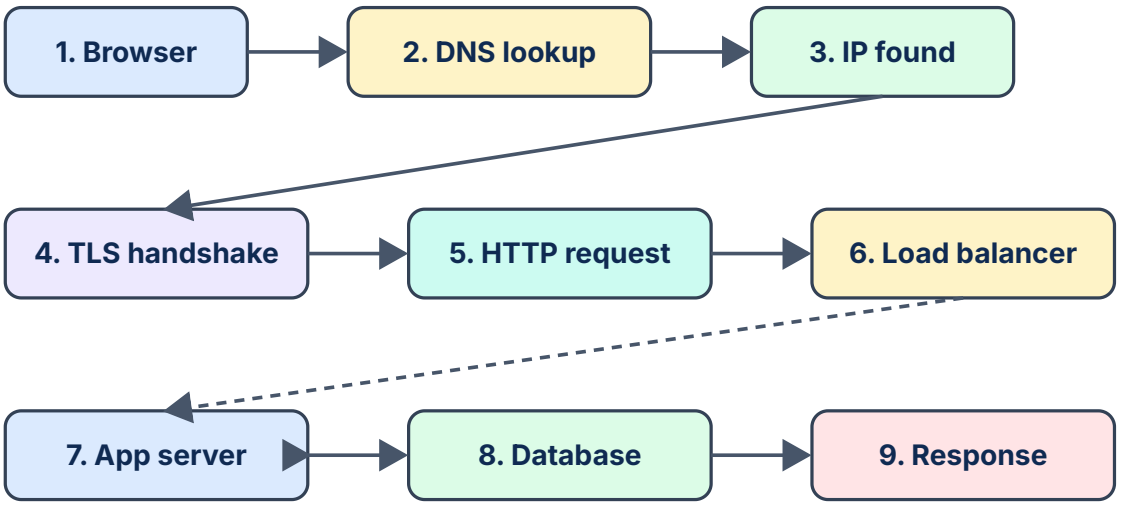
The classic beginner bill shock: a forgotten VM, a NAT gateway or a public IP left running for a month. When a lab ends, **delete everything you created**.

Networking Basics

Every cloud service rides on ordinary internet networking. Five ideas cover most of it.

Idea	Plain meaning	Example
IP address	Street address of a machine	`203.0.113.25` (IPv4), `2001:db8::1` (IPv6)
Port	Door number on that machine	80 HTTP, 443 HTTPS, 22 SSH, 5432 PostgreSQL
DNS	Phone book mapping names to IPs	`example.com` becomes `203.0.113.25`
Protocol	Agreed language of the conversation	TCP, UDP, HTTP, TLS
HTTP	Request/response rules of the web	`GET /index.html`

What happens when you open a website



The journey of one page view

TRY IT

Try it: run `nslookup example.com` or `dig example.com` in a terminal and read the answer section. That is step 2 above.

KEY IDEA

Latency is delay (milliseconds). **Bandwidth** is volume per second. A road can be wide (high bandwidth) yet long (high latency).

CIDR, VPC and Subnets

A **VPC** (Virtual Private Cloud, called a Virtual Network on Azure) is your own isolated network inside the provider. You choose its address range using **CIDR** notation.

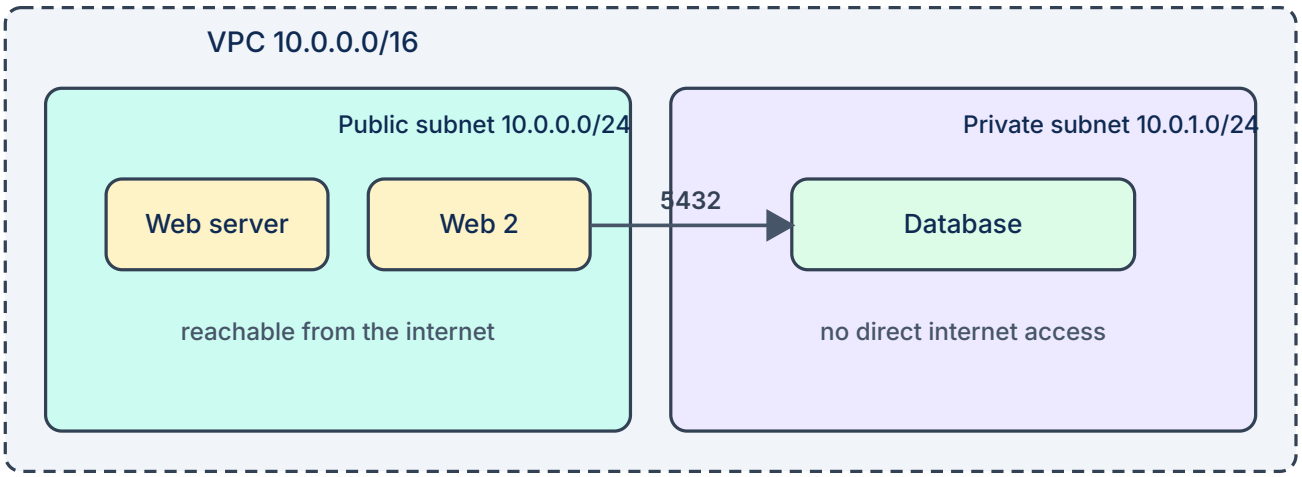
`10.0.0.0/16` means: the first **16 bits** are fixed (the network), the remaining 16 bits are free for hosts. The bigger the slash number, the smaller the network.

cidr_demo.py

```
import ipaddress

vpc = ipaddress.ip_network("10.0.0.0/16")
print("VPC", vpc, "has", vpc.num_addresses, "addresses")
public, private = list(vpc.subnets(new_prefix=24))[:2]
print("public subnet ", public, "→", public.num_addresses, "addresses")
print("private subnet", private, "→", private.num_addresses, "addresses")
print("10.0.1.25 inside private?", ipaddress.ip_address("10.0.1.25") in private)
```

```
VPC 10.0.0.0/16 has 65536 addresses
public subnet  10.0.0.0/24 → 256 addresses
private subnet 10.0.1.0/24 → 256 addresses
10.0.1.25 inside private? True
```



Web tier public, data tier private

WATCH OUT

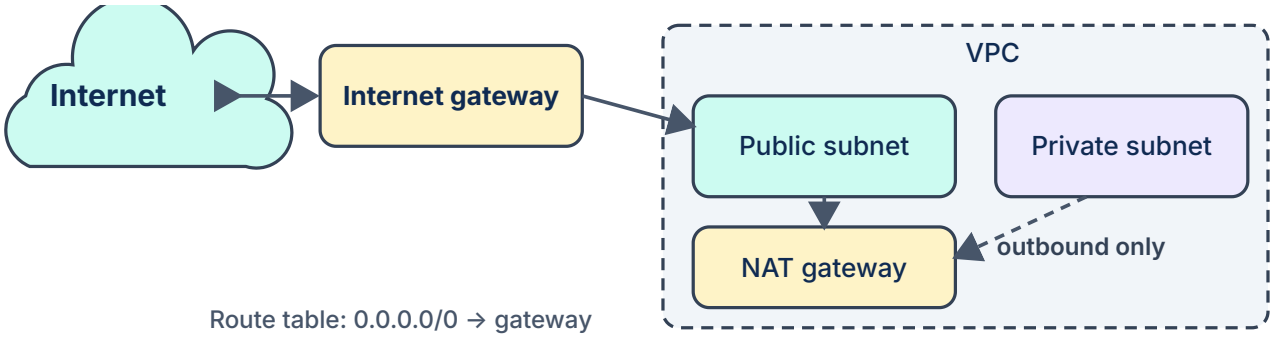
Never let two networks you may ever connect use **overlapping CIDR ranges**. Plan address space up front: it is painful to change later.

NOTE

Cloud providers reserve a few addresses in every subnet (network, gateway, DNS and so on), so a /24 gives slightly fewer usable hosts than 256.

Gateways, Routes and Firewalls

Subnets alone do not decide who can reach what. **Route tables, gateways and firewall rules** do.



Private servers can download updates through the NAT gateway, but nobody on the internet can start a connection to them.

Gateways decide traffic paths

Component	Job
Internet gateway	Door between the VPC and the public internet
NAT gateway	Lets private servers reach out (updates) without being reachable in
Route table	Rules: destination range → where to send it
Security group	Virtual firewall on a server; *stateful*, allow-rules only
Network ACL	Firewall on a subnet; *stateless*, allow and deny rules

A sensible security-group set

Group	Inbound rule	Source
web-sg	TCP 443 (HTTPS)	Anywhere (0.0.0.0/0)
web-sg	TCP 22 (SSH)	Your office IP only
db-sg	TCP 5432	web-sg only (not the internet)

WATCH OUT

Never open SSH (22) or a database port to `0.0.0.0/0`. Automated scanners find open ports within minutes.

Storage Types and Databases

There are three storage shapes. Choosing the right one is half of cloud design.

Type	Looks like	Best for	Examples
Block	A raw disk attached to one VM	OS disks, databases	EBS, Azure Disk, Persistent Disk
File	A shared network drive	Shared files between servers	EFS, Azure Files, Filestore
Object	Files in buckets, accessed by URL/API	Images, backups, logs, websites, data lakes	S3, Blob Storage, Cloud Storage



Object storage: you ask for a key, not a drive letter

Managed databases

Running a database yourself means patching, backups and failover. **Managed databases** (RDS, Azure SQL, Cloud SQL) do that for you.

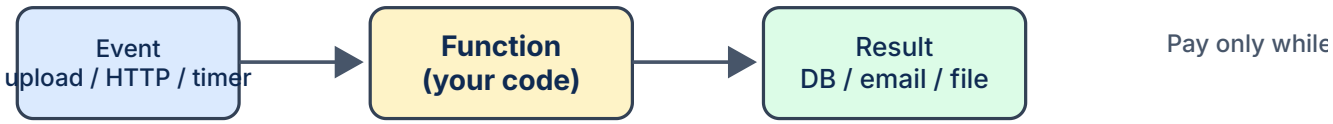
	Relational (SQL)	NoSQL
Shape	Tables, rows, fixed schema	Documents, key-value, wide-column, graph
Strength	Joins, transactions, consistency	Huge scale, flexible schema, speed
Examples	PostgreSQL, MySQL	DynamoDB, Cosmos DB, Firestore, Redis
Pick when	Orders, payments, accounts	Sessions, catalogues, feeds, caching

PRO TIP

Default advice: start with a managed **relational** database. Move specific workloads to NoSQL when you have a measured reason.

Serverless, Recap and Mini-Lab

Serverless does not mean no servers. It means *you* do not manage them: you upload a function, the provider runs it when an event arrives and bills per execution.



Typical serverless flow

Good fit	Poor fit
Image resizing after upload, webhooks, scheduled jobs, light APIs	Long-running jobs, constant heavy load, tight latency needs where cold starts hurt

Chapter recap

- ☐ Cloud = rented, on-demand, measured computing over the internet
- ☐ IaaS gives machines, PaaS gives a platform, SaaS gives an app
- ☐ Regions contain zones; use two or more zones for availability
- ☐ Scale out with load balancers; keep servers stateless
- ☐ RTO and RPO set your disaster-recovery budget
- ☐ Shared responsibility: you secure what you put in the cloud
- ☐ VPC, subnets, gateways and security groups control traffic
- ☐ Object storage for files, managed relational DB for records

⚒ Hands-on lab: Design on paper

1. Pick an app you know (a blog, a college portal, a food-order app).
2. Draw a VPC with one public and one private subnet in two zones.
3. Place a load balancer, two web servers, one database and one object-storage bucket.
4. Write your RTO and RPO and the DR strategy you would choose.
5. List three security-group rules and one IAM policy you would create.

The Big Three

02 CHAPTER 2 AWS, Azure and Google Cloud

Three clouds, one set of ideas

WHAT YOU WILL LEARN

- How the three providers organise accounts and resources
- Equivalent services across all three
- Hands-on walk-throughs with each command line
- IAM, monitoring and how to choose a cloud

Amazon Web Services (AWS), **Microsoft Azure** and **Google Cloud Platform (GCP)** are the three largest public clouds. Each sells hundreds of services, but they all rest on the same building blocks you met in Chapter 1.

	AWS	Azure	Google Cloud
Started	2006	2010	2008 (App Engine), broad IaaS later
Known for	Widest catalogue, huge community	Enterprise, Microsoft 365 and .NET integration	Data, analytics, Kubernetes, AI
Top-level container	Account (inside an Organization)	Tenant > Subscription	Organization > Folder > Project
Grouping resources	Tags, CloudFormation stacks	Resource group	Project
CLI	`aws`	`az`	`gcloud`
IaC (native)	CloudFormation, CDK	Bicep, ARM templates	Deployment Manager, Config Connector

KEY IDEA

The vocabulary changes; the **concepts do not**. A VM is a VM, a bucket is a bucket. Learn one cloud well and you can read the other two within days.

Service Map (1): Compute and Storage

Use this page as a translation table. The left column is what you want; the others are what each cloud calls it.

Need	AWS	Azure	Google Cloud
Virtual machines	EC2	Virtual Machines	Compute Engine
Auto scaling groups	EC2 Auto Scaling	VM Scale Sets	Managed Instance Groups
Simple platform for web apps	Elastic Beanstalk, App Runner	App Service	App Engine, Cloud Run
Containers (managed)	ECS, Fargate	Container Apps, ACI	Cloud Run
Kubernetes	EKS	AKS	GKE
Functions	Lambda	Azure Functions	Cloud Run functions
Batch jobs	AWS Batch	Azure Batch	Cloud Batch

Storage need	AWS	Azure	Google Cloud
Object storage	S3	Blob Storage	Cloud Storage
Block disk	EBS	Managed Disks	Persistent Disk
Shared file system	EFS, FSx	Azure Files	Filestore
Archive / cold data	S3 Glacier	Blob Archive tier	Archive storage class
Backup service	AWS Backup	Azure Backup	Backup and DR Service
CDN	CloudFront	Azure Front Door / CDN	Cloud CDN

NOTE

Product names change as providers rebrand and reorganise services. When in doubt, search for the **concept** plus the cloud name, and read the current documentation.

PRO TIP

Interview favourite: *"Where would you store user-uploaded photos?"* Answer: **object storage** (S3 / Blob / Cloud Storage), keep only the file URL in the database.

Service Map (2): Data, Network, Security

Database need	AWS	Azure	Google Cloud
Managed relational	RDS, Aurora	Azure SQL, Database for PostgreSQL/MySQL	Cloud SQL, AlloyDB
Key-value / document NoSQL	DynamoDB	Cosmos DB	Firestore, Bigtable
In-memory cache	ElastiCache	Azure Cache for Redis	Memorystore
Data warehouse	Redshift	Synapse Analytics, Fabric	BigQuery
Messaging / queues	SQS, SNS	Service Bus, Event Grid	Pub/Sub

Network need	AWS	Azure	Google Cloud
Private network	VPC	Virtual Network (VNet)	VPC network
Load balancer	Elastic Load Balancing	Load Balancer, App Gateway	Cloud Load Balancing
DNS	Route 53	Azure DNS	Cloud DNS
VPN / private link	Site-to-Site VPN, Direct Connect	VPN Gateway, ExpressRoute	Cloud VPN, Interconnect
Firewall rules	Security Groups, NACLs	Network Security Groups	VPC firewall rules

Security need	AWS	Azure	Google Cloud
Identity and access	IAM	Entra ID + Azure RBAC	Cloud IAM
Secrets	Secrets Manager	Key Vault	Secret Manager
Encryption keys	KMS	Key Vault	Cloud KMS
Threat detection	GuardDuty	Defender for Cloud	Security Command Center

KEY IDEA

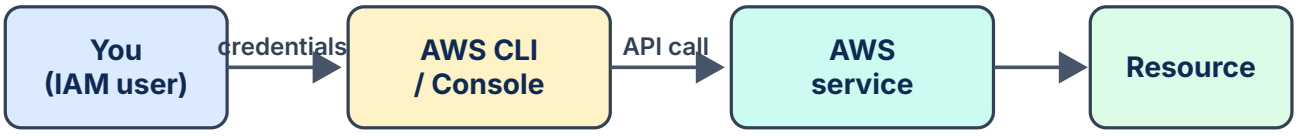
Notice the pattern: every cloud has a **queue/topic service**, a **secrets service** and a **key service**. Learn the pattern, not the brand name.

AWS: Account, Console, CLI and IAM

Everything in AWS lives inside an **account**, and the account has one all-powerful **root user** (the email you signed up with). Your first job is to stop using it.

1. Sign up, then open **IAM** and turn on **MFA** for the root user.
2. Create an **IAM user** (or IAM Identity Center user) for yourself with the permissions you need. Do not use root again.
3. Create a **billing budget** with an email alert in the Billing console.
4. Install the **AWS CLI** and connect it to your user.

```
terminal
# one-time setup (illustrative: needs your own AWS account)
$ aws configure
# AWS Access Key ID [None]: <your key id>
# AWS Secret Access Key [None]: <your secret>
# Default region name [None]: ap-south-1
# Default output format [None]: json
$ aws sts get-caller-identity
```



Every click or command becomes an authenticated API call

PRO TIP

Better than long-lived keys: **IAM Identity Center** / `aws sso login`, or **roles** that give temporary credentials. If you must create access keys for learning, delete them when finished.

WATCH OUT

If an access key ever appears in a public repo, treat it as stolen: deactivate it immediately, then rotate.

AWS: Launch Your First Server (EC2)

EC2 (Elastic Compute Cloud) rents virtual machines. The console wizard asks for six things:

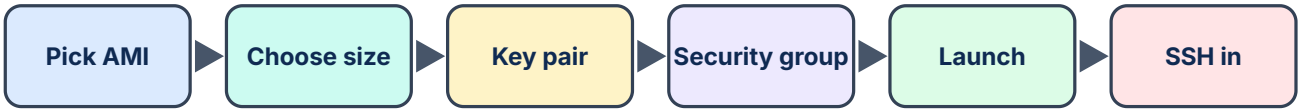
Setting	What it is	Beginner choice
AMI	Machine image: the OS template	Amazon Linux or Ubuntu
Instance type	Size: vCPU + RAM	Smallest free-tier eligible type
Key pair	SSH login key	Create new, download `.pem` once
Network	VPC and subnet	Default VPC is fine for a lab
Security group	Firewall	SSH from *your IP* only
Storage	Root disk size	Default

launch

```
# illustrative CLI version of the same launch
$ aws ec2 run-instances \
  --image-id ami-0123456789abcdef0 \
  --instance-type t3.micro \
  --key-name my-key \
  --security-group-ids sg-0abc1234 \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=demo}]'
$ aws ec2 describe-instances --query "Reservations[].Instances[].State.Name"
```

connect (Amazon Linux user is ec2-user; Ubuntu uses ubuntu)

```
$ chmod 400 my-key.pem
$ ssh -i my-key.pem ec2-user@<public-ip>
$ sudo dnf update -y
```



The six-step wizard

WATCH OUT

Stop keeps the disk (small storage charge); **Terminate** deletes the instance. When your lab is over, terminate it.

NOTE

The AMI ID shown above is a placeholder. Real AMI IDs differ per region; copy one from the console.

AWS: Object Storage with S3

S3 stores objects (files) in **buckets**. Bucket names are **globally unique**, objects are addressed by a **key** such as `photos/2026/cat.jpg`. S3 is designed for extremely high durability.

```
S3 in six commands (illustrative)
$ aws s3 mb s3://my-unique-notes-bucket-2026
$ aws s3 cp notes.pdf s3://my-unique-notes-bucket-2026/docs/
$ aws s3 ls s3://my-unique-notes-bucket-2026/docs/
$ aws s3 sync ./site s3://my-unique-notes-bucket-2026/site
$ aws s3 presign s3://my-unique-notes-bucket-2026/docs/notes.pdf --expires-in
600
$ aws s3 rb s3://my-unique-notes-bucket-2026 --force
```

Feature	What it gives you
Storage classes	Standard, Infrequent Access, Glacier archive tiers: cheaper if you read less
Versioning	Keep old versions; recover from accidental delete
Lifecycle rules	Auto-move or delete objects after N days
Presigned URL	Temporary link to a private file
Static website hosting	Serve HTML straight from a bucket
Encryption	On by default; choose keys via KMS

WATCH OUT

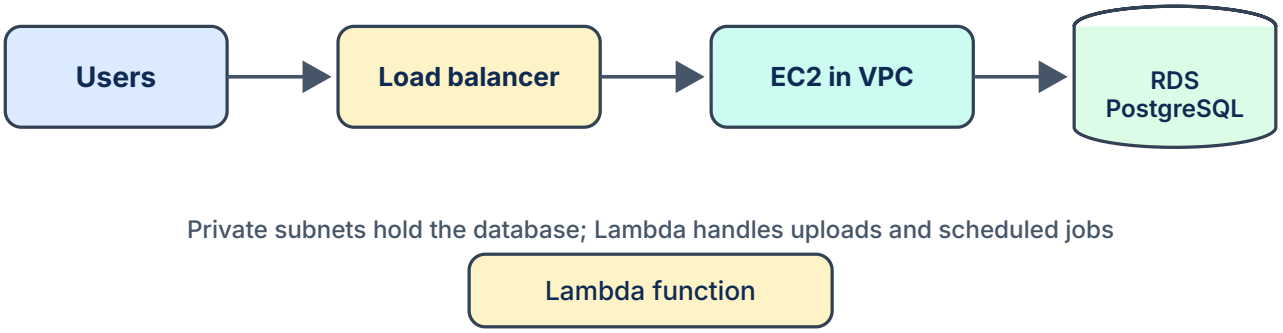
Buckets are **private by default**. Keep *Block Public Access* switched on unless you are intentionally hosting a public website. Misconfigured public buckets cause many real data leaks.

KEY IDEA

Remember the shape: **bucket name + key = object**. There are no real folders; slashes in the key just look like folders.

AWS: VPC, RDS and Lambda

Three more services complete a typical AWS web stack.



A classic three-part AWS stack

- **VPC**: your private network (Chapter 1). Default VPC exists for quick labs; build a custom one for real projects.
- **RDS**: managed PostgreSQL, MySQL and others. Handles backups, patching, Multi-AZ failover.
- **Lambda**: run a function in response to an event. You pay per request and run time.

app.py (a Lambda behind an HTTP endpoint)

```
import json

def lambda_handler(event, context):
    name = (event.get("queryStringParameters") or {}).get("name", "world")
    return {"statusCode": 200,
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps({"message": f"Hello, {name}!"})}
```

```
{'statusCode': 200, 'headers': {'Content-Type': 'application/json'}, 'body':
'{"message": "Hello, Asha!"}'}
200
```

deploy (illustrative)

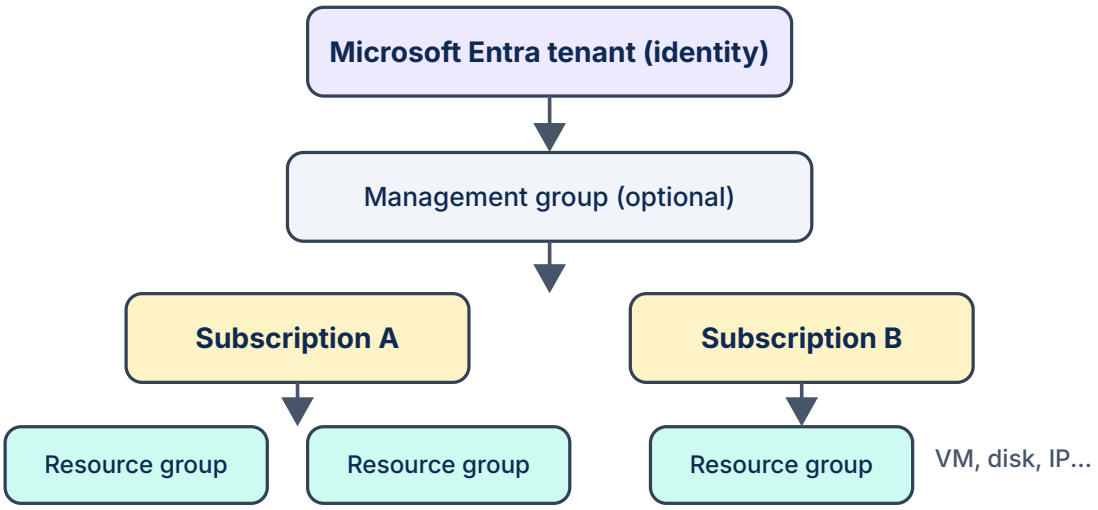
```
$ zip function.zip app.py
$ aws lambda create-function --function-name hello --runtime python3.12 \
  --role arn:aws:iam::123456789012:role/lambda-basic \
  --handler app.lambda_handler --zip-file fileb://function.zip
$ aws lambda invoke --function-name hello out.json
```

PRO TIP

The handler above was run locally with a fake event. This is the easiest way to test a function before deploying.

Azure: Concepts and Structure

Azure organises everything in a clear hierarchy. Understanding it avoids 90% of beginner confusion.



Tenant > subscription > resource group > resources

Term	Meaning
Tenant	Your organisation's identity directory (Microsoft Entra ID)
Subscription	Billing and access boundary; costs roll up here
Resource group	A folder of related resources with the same lifecycle
Resource	One item: VM, storage account, database
Region	Where the resource physically lives

PRO TIP

Resource groups are Azure's superpower for cleanup: **delete the group, delete everything inside it**. Put every lab in its own group.

KEY IDEA

Azure access is **RBAC**: a *role* (Reader, Contributor, Owner) is assigned to a *principal* at a *scope* (subscription, group or resource).

Azure: VMs, Blob Storage and App Service

Virtual machine (illustrative)

```
$ az login
$ az group create --name demo-rg --location centralindia
$ az vm create --resource-group demo-rg --name demo-vm \
  --image Ubuntu2204 --admin-username azureuser --generate-ssh-keys
$ az vm open-port --resource-group demo-rg --name demo-vm --port 80
```

Blob storage

```
$ az storage account create --name demostore2026x --resource-group demo-rg \
  --sku Standard_LRS --location centralindia
$ az storage container create --account-name demostore2026x --name files --auth-
mode login
$ az storage blob upload --account-name demostore2026x --container-name files \
  --name notes.pdf --file notes.pdf --auth-mode login
```

App Service, then clean up

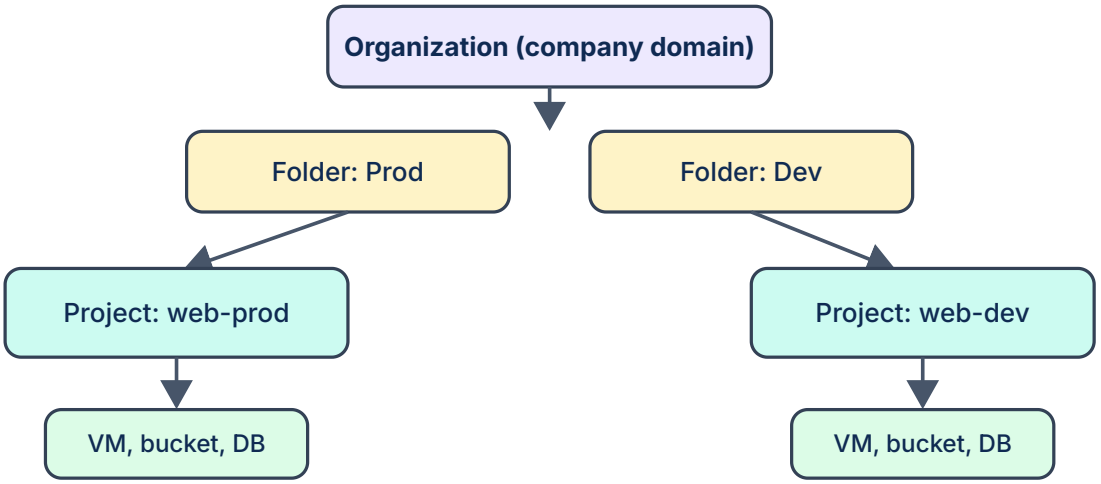
```
$ az webapp up --name demo-flask-2026 --resource-group demo-rg \
  --runtime "PYTHON:3.12" --sku B1
$ az group delete --name demo-rg --yes --no-wait
```

Azure service	Think of it as
Virtual Machines	EC2: rent a server
Storage account > Blob	S3: object storage (account name must be globally unique)
App Service	Elastic Beanstalk: push code, platform runs it
Azure Functions	Lambda: event-driven code

> **NOTE**
> Redundancy codes such as **LRS** (copies in one datacentre), **ZRS** (across zones) and **GRS** (to a paired region) are Azure's way of choosing durability versus cost.

Google Cloud: Concepts and Structure

Google Cloud's unit of everything is the **project**: it holds resources, billing links, APIs and permissions.



Organization > folders > projects > resources

Term	Meaning
Project ID	Globally unique name, used by `gcloud` and APIs
API enablement	Services must be switched on per project (e.g. Compute Engine API)
Billing account	Linked to projects; one account can pay for many projects
Service account	A non-human identity for apps and pipelines
Region / zone	`asia-south1` (Mumbai), `asia-south1-a` is one zone in it

first steps (illustrative)

```
$ gcloud auth login
$ gcloud projects create demo-lab-2026 --name="Demo Lab"
$ gcloud config set project demo-lab-2026
$ gcloud services enable compute.googleapis.com run.googleapis.com
```

PRO TIP

Use **separate projects** for dev, test and production. Deleting a project removes every resource inside it, which makes cleanup easy.

Google Cloud: Compute, Storage and Cloud Run

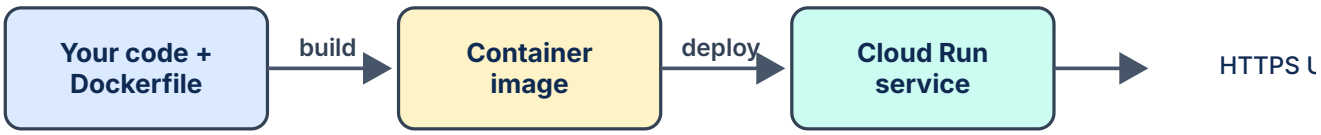
```
Compute Engine (illustrative)
$ gcloud compute instances create demo-vm --zone=asia-south1-a \
  --machine-type=e2-micro --image-family=debian-12 --image-project=debian-cloud
$ gcloud compute ssh demo-vm --zone=asia-south1-a
$ gcloud compute instances delete demo-vm --zone=asia-south1-a
```

```
Cloud Storage
$ gcloud storage buckets create gs://demo-notes-2026 --location=asia-south1
$ gcloud storage cp notes.pdf gs://demo-notes-2026/docs/
$ gcloud storage ls gs://demo-notes-2026/docs/
```

Cloud Run: containers without servers

Cloud Run runs a container and scales it to zero when nobody visits. It is a very friendly first step into containers, which is why Chapter 4 follows.

```
deploy from source (needs a Dockerfile or supported language)
$ gcloud run deploy hello --source . --region asia-south1 --allow-
unauthenticated
```



From source to a public URL

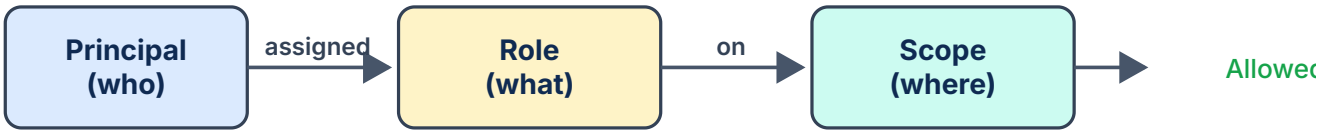
WATCH OUT

`--allow-unauthenticated` makes the URL public. That is right for a demo website and wrong for an internal API.

IAM Compared

All three clouds answer the same question: **who** can do **what** on **which** resource.

	AWS IAM	Azure RBAC	Google Cloud IAM
Who	Users, groups, roles	Users, groups, service principals, managed identities	Google accounts, groups, service accounts
What	Policy (JSON allow/deny)	Role definition (Reader, Contributor, Owner, custom)	Role (Viewer, Editor, Owner, predefined, custom)
Where	Resource ARN in the policy	Scope: subscription / group / resource	Resource hierarchy: org / folder / project / resource
App identity	IAM role for EC2 or Lambda	Managed identity	Service account attached to the service



The universal permission sentence

Rules that apply on every cloud

- ☐ Humans use SSO + MFA; no shared logins
- ☐ Apps use roles / managed identities / service accounts, never a person's key
- ☐ Start with read-only, add permissions when something fails
- ☐ Review who has *Owner* every month
- ☐ Log every administrative action (CloudTrail, Activity Log, Audit Logs)

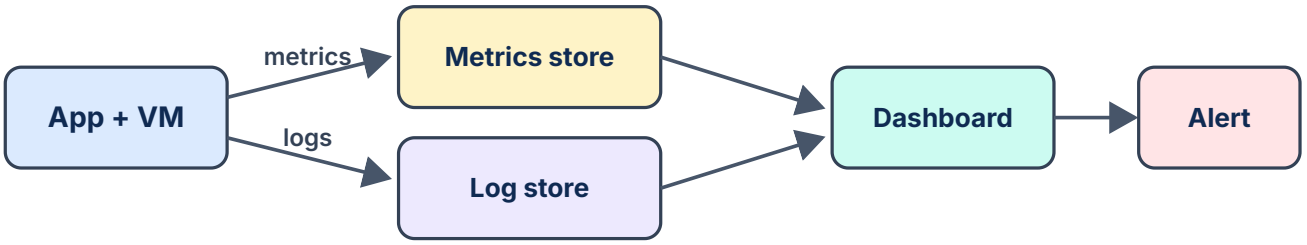
INTERVIEW ANSWER

If someone asks *"how do you give an EC2 server access to S3 safely?"*, the answer is **attach a role to the server**, not paste keys onto it.

Monitoring and Logging Compared

Once an app is live you must see what it is doing. Monitoring has three pillars: **metrics** (numbers over time), **logs** (event text) and **traces** (a request's path across services).

Pillar	AWS	Azure	Google Cloud
Metrics and alarms	CloudWatch Metrics + Alarms	Azure Monitor Metrics + Alerts	Cloud Monitoring
Logs	CloudWatch Logs	Log Analytics	Cloud Logging
Traces	X-Ray	Application Insights	Cloud Trace
Audit of who did what	CloudTrail	Activity Log	Cloud Audit Logs
Dashboards	CloudWatch dashboards	Azure dashboards, Workbooks	Monitoring dashboards



Collect, store, visualise, alert

An alert worth having

```
alert definition in plain words
Name:      High CPU on web servers
Condition: average CPU > 80% for 5 minutes
Notify:    on-call email + chat channel
Action:    auto scaling adds a server
```

PRO TIP

Alert on **symptoms users feel** (errors, slowness) before **causes** (CPU). Too many noisy alerts train people to ignore all of them.

NOTE

Chapter 5 returns to observability and SLOs in more depth.

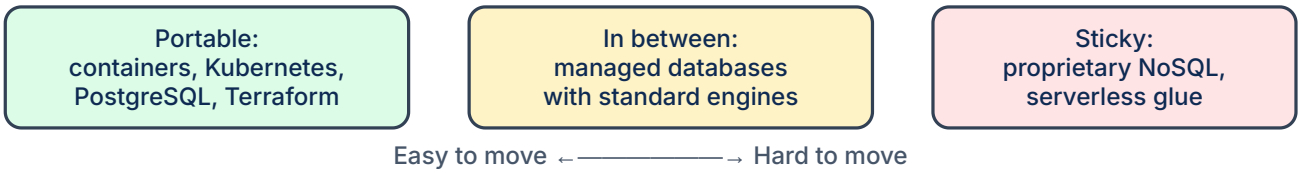
Choosing a Cloud and Avoiding Lock-in

There is no universally best cloud. Pick using your situation, not hype.

If you…	Consider
Already use Microsoft 365, Windows servers, .NET or Active Directory	**Azure**
Want the broadest service choice and the biggest community and job market	**AWS**
Care most about data analytics, Kubernetes or ML tooling	**Google Cloud**
Need to run in a country or on a platform one provider covers best	Whichever covers it
Are learning	Whichever has the best free-tier credit and tutorials for you

What is vendor lock-in?

Lock-in is how costly it is to leave. The more you use provider-specific services, the harder the move. It is a *trade-off*, not a sin: proprietary services are often cheaper and faster to adopt.



A lock-in spectrum

Practical hedges

- Package apps in **containers** (Chapter 4)
- Describe infrastructure with **Terraform** (Chapter 5)
- Prefer **standard engines** (PostgreSQL, Redis) even when managed
- Keep data export paths documented

PRO TIP

Do not over-engineer for a move you may never make. Decide how much lock-in is acceptable, write it down, and review yearly.

Free Tiers, Budgets and Safe Cleanup

Free tiers and trial credits are generous but limited and they change. **Check each provider's current free-tier page** before you start, and note when your credits expire.

Your safety net

1. Create a **budget** with an alert at a low amount (AWS Budgets, Azure Cost Management, Google Cloud Billing budgets).
2. Enable **MFA** on every admin identity.
3. Tag or group every lab by name: `lab-ch2-ec2`.
4. Prefer the **smallest** size and a **single region**.
5. End each session with a **cleanup check**.

End-of-lab cleanup checklist

Look for	AWS	Azure	Google Cloud
Running VMs	EC2 instances	Virtual Machines	Compute Engine instances
Orphan disks	EBS volumes	Disks	Persistent Disks
Public IPs	Elastic IPs	Public IP addresses	External IP addresses
Gateways and balancers	NAT gateways, load balancers	App Gateways, NAT	Cloud NAT, load balancers
Everything at once	Delete resources or close lab account	**Delete resource group**	**Delete project**

PRO TIP

A tip that saves money: treat cloud labs like cooking. **Clean the kitchen before you leave**, every time.

WATCH OUT

Deleting the wrong project or group is also irreversible. Double-check the name before you confirm.

Chapter 2 Recap, Certifications and Lab

Recap

- ☐ Accounts, subscriptions and projects are the top-level containers
- ☐ Service names differ, concepts are shared
- ☐ Use CLIs for repeatable work: `aws`, `az`, `gcloud`
- ☐ IAM = principal + role + scope; apps use roles, not keys
- ☐ Monitoring = metrics + logs + traces + alerts
- ☐ Delete lab resources the same day

Entry-level certification paths

Cloud	Starting exam	Then
AWS	Cloud Practitioner	Solutions Architect Associate, Developer Associate, SysOps / DevOps
Azure	AZ-900 Azure Fundamentals	AZ-104 Administrator, AZ-204 Developer
Google Cloud	Cloud Digital Leader	Associate Cloud Engineer

PRO TIP

Certifications open doors, but **hands-on projects** (Chapter 6) are what you will talk about in interviews. Do both.

⚒ Hands-on lab: Same app, one cloud, end to end

1. Create a free account on one cloud and enable MFA plus a budget alert.
2. Launch the smallest VM, SSH in, and run `uname -a`.
3. Create a bucket/container, upload a file, and open it with a presigned or signed link.
4. Write down the equivalent service names on the other two clouds using the service maps.
5. Delete everything and confirm the bill dashboard shows no running resources.

Why Linux?

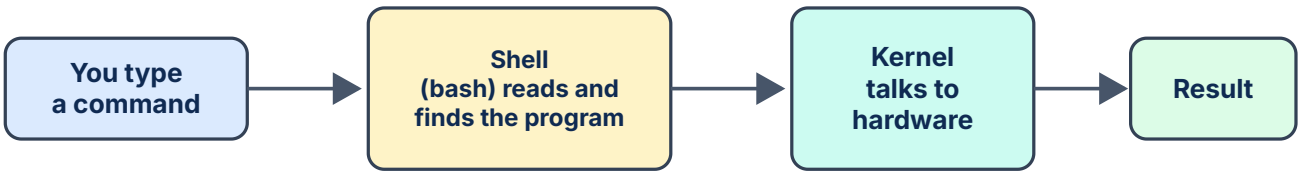
03 CHAPTER 3 Linux and Git Fundamentals

The two tools every cloud engineer uses daily

WHAT YOU WILL LEARN

- Move around Linux, manage files, permissions and processes
- Filter text and chain commands with pipes
- Write and schedule shell scripts
- Track changes, branch, merge and collaborate with Git

Most servers in the cloud run **Linux**. When you SSH into a VM, deploy a container or read a CI log, you are in a Linux shell. Linux is free, stable, scriptable and the default platform for containers.



Shell, kernel and hardware

Distro family	Examples	Package tool
Debian family	Ubuntu, Debian	`apt`
Red Hat family	RHEL, Fedora, Amazon Linux, Rocky	`dnf` / `yum`
Alpine	Alpine (tiny, used in containers)	`apk`

KEY IDEA

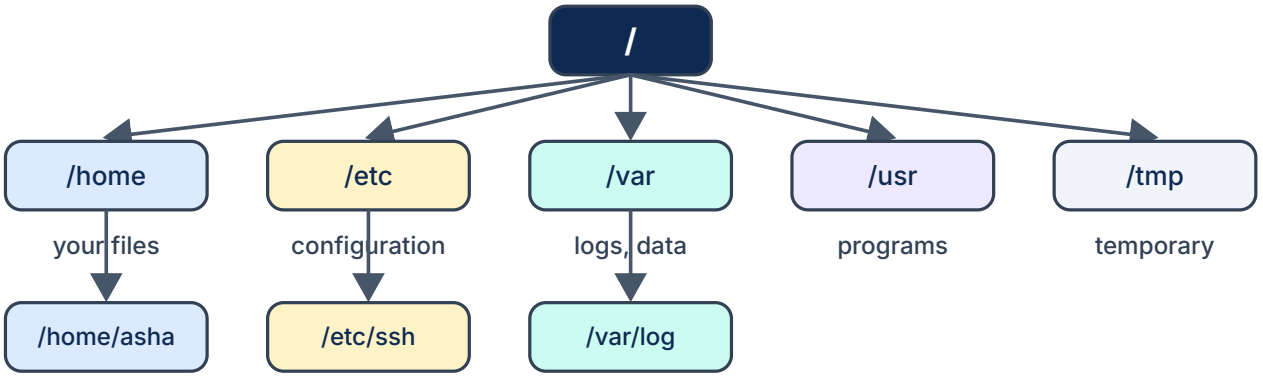
Linux commands follow one pattern: **command -options arguments**. Example: `ls -l /etc` runs *ls*, with option *-l*, on the argument */etc*.

PRO TIP

Use `man ls` or `ls --help` to read documentation offline. Searching docs is a real skill, not cheating.

The Linux Filesystem

Linux has **one tree** that starts at `/` (root). There are no drive letters. Everything, even hardware and running processes, appears as a file or folder.



Also: /bin and /usr/bin (commands), /opt (add-on software), /dev (devices), /proc (process info)

The directories you will meet

Path	What lives there
`/home/<user>`	Personal files (`~` is shorthand for this)
`/etc`	System configuration files
`/var/log`	Log files: first place to look when something breaks
`/tmp`	Temporary files, wiped on reboot
`/usr/bin` , `/bin`	Programs

KEY IDEA

Absolute paths start with `/` (`/etc/hosts`). **Relative** paths start from where you are (`../notes`). `.` is here, `..` is the parent.

Navigating and Looking Around

Seven commands cover most navigation. Try them in a scratch folder.

navigation

```
$ pwd
$ ls
$ ls -la projects
$ cd projects/web
$ pwd
$ cd ..
$ cd ~
$ tree -L 2 projects 2>/dev/null || find projects | sort
```

```
/home/user
notes
projects
total 12
drwxr-xr-x 3 user user 4096 Nov 11 06:43 .
drwx------ 4 user user 4096 Nov 11 06:43 ..
drwxr-xr-x 4 user user 4096 Nov 11 06:43 web
/home/user/projects/web
projects
projects/web
projects/web/css
projects/web/css/site.css
projects/web/index.html
projects/web/js
projects/web/js/app.js
```

Command	Does
`pwd`	Print working directory (where am I?)
`ls -la`	List all files, long format, including hidden
`cd dir`	Change directory; `cd ..` up; `cd -` back
`find . -name '*.js'`	Search by name
`which python3`	Where is this program?

PRO TIP

Press **Tab** to autocomplete and the **Up arrow** or `Ctrl+R` to search history. Experienced engineers type very little.

Working with Files

create, copy, move, delete

```
$ mkdir -p lab/docs
$ cd lab
$ touch a.txt
$ echo "first line" > docs/note.txt
$ echo "second line" >> docs/note.txt
$ cp docs/note.txt docs/backup.txt
$ mv a.txt docs/renamed.txt
$ ls docs
$ cat docs/note.txt
$ rm docs/backup.txt
$ ls docs
```

```
backup.txt
note.txt
renamed.txt
first line
second line
note.txt
renamed.txt
```

Command	Meaning	Danger level
`mkdir -p a/b`	Make folders (and parents)	Safe
`touch f`	Create empty file / update time	Safe
`cp -r src dst`	Copy (recursive for folders)	Safe
`mv a b`	Move or rename	Overwrites target!
`rm f`, `rm -r dir`	Delete: **no recycle bin**	High

viewing parts of a file

```
$ seq 1 20 > numbers.txt
$ head -n 3 numbers.txt
$ wc -l numbers.txt
```

```
1
2
3
20 numbers.txt
```

WATCH OUT

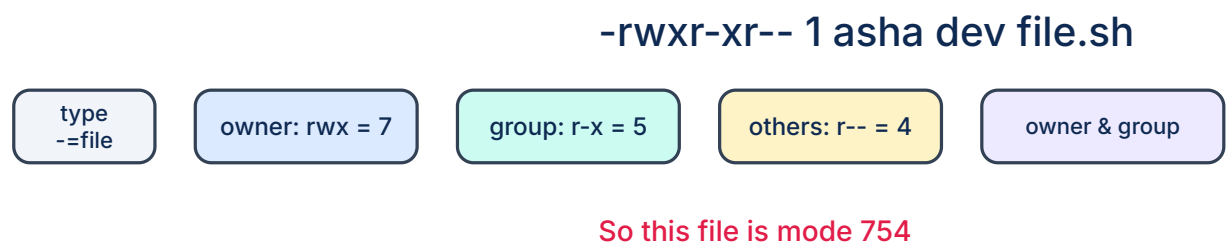
Never run `rm -rf /` or `rm -rf` with an unchecked variable (`rm -rf $DIR/`). If `$DIR` is empty you may delete far more than you intended. Check with `ls` first.

Permissions and Ownership

Every file has an **owner**, a **group** and three permission sets: for the owner (**u**), the group (**g**) and everyone else (**o**). Each set has **r**ead (4), **w**rite (2) and e**x**ecute (1).

reading and changing permissions

```
$ touch deploy.sh
$ chmod 644 deploy.sh
$ ls -l deploy.sh
$ chmod 755 deploy.sh
$ ls -l deploy.sh
$ chmod u=rw,go= deploy.sh
$ ls -l deploy.sh
-rw-r--r-- 1 user user 0 Nov 11 06:43 deploy.sh
-rwxr-xr-x 1 user user 0 Nov 11 06:43 deploy.sh
-rw------- 1 user user 0 Nov 11 06:43 deploy.sh
```



Anatomy of ls -l

Mode	Meaning	Typical use
`644`	owner rw, others read	Normal files
`755`	owner rwx, others rx	Scripts, programs, directories
`600`	owner only	SSH private keys, secrets
`400`	owner read only	The `.pem` key you download

PRO TIP

SSH refuses a private key that others can read. If you see *"UNPROTECTED PRIVATE KEY FILE"*, the fix is `chmod 400 key.pem`.

WATCH OUT

`chmod 777` is almost never the fix. For admin tasks use `sudo cmd` (one command as root) and keep daily work as a normal user. Never run application servers as root.

Reading and Searching Text

Logs, configs and CSV files are text. These tools find what matters fast.

grep: find lines that match

```
$ cat > app.log <<'LOG'
$ 2026-11-11 10:00:01 INFO  user=asha action=login
$ 2026-11-11 10:00:07 ERROR db timeout after 30s
$ 2026-11-11 10:01:15 INFO  user=ravi action=login
$ 2026-11-11 10:02:44 ERROR payment failed order=88
$ 2026-11-11 10:03:02 WARN  disk 85% full
$ LOG
$ grep ERROR app.log
$ grep -c INFO app.log
$ grep -v INFO app.log
$ grep -i "user=" app.log | wc -l
```

```
2026-11-11 10:00:07 ERROR db timeout after 30s
2026-11-11 10:02:44 ERROR payment failed order=88
2
2026-11-11 10:00:07 ERROR db timeout after 30s
2026-11-11 10:02:44 ERROR payment failed order=88
2026-11-11 10:03:02 WARN  disk 85% full
2
```

Tool	Job	Example
`grep`	Find matching lines	`grep -r 'TODO' src/`
`sed`	Replace text in a stream	`sed 's/foo/bar/g' f`
`awk`	Work with columns	`awk '{print $1}' f`
`sort`, `uniq`	Order and count unique	`sort f | uniq -c`
`cut`	Pick fields	`cut -d, -f2 f.csv`

which log level is most common?

```
$ awk '{print $3}' app.log | sort | uniq -c | sort -rn
```

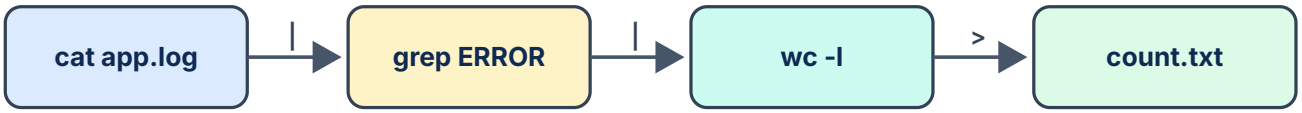
```
      2 INFO
      2 ERROR
      1 WARN
```

PRO TIP

`grep -r --include='*.py' 'password' .` is a quick audit for hard-coded secrets in a project.

Pipes and Redirection

Programs read from **stdin**, write to **stdout** and report problems on **stderr**. You can wire them together.



A pipeline: each program does one small job

Symbol	Meaning
`|`	Send stdout of the left command into stdin of the right
`>`	Write stdout to a file (overwrite)
`>>`	Append stdout to a file
`2>`	Write stderr to a file
`&>` or `2>&1`	Combine stdout and stderr
`&&` / `||`	Run next only if previous succeeded / failed

redirect and chain

```
$ printf 'ERROR a\nINFO b\nERROR c\n' > app.log
$ grep ERROR app.log | wc -l > count.txt
$ cat count.txt
$ ls missing_file 2> errors.txt
$ cat errors.txt
$ mkdir build && echo "created build"
$ false || echo "previous command failed"
```

```
2
ls: cannot access 'missing_file': No such file or directory
created build
previous command failed
```

KEY IDEA

`command && next` is the building block of every deployment script: *only continue if the last step worked.*

Processes, Services and systemd

A running program is a **process** with a numeric **PID**. Long-running background programs (web servers, databases) are **services** managed by **systemd** on most distros.

Command	Use
`ps aux`	List all processes
`top` / `htop`	Live view of CPU and memory
`kill PID`, `kill -9 PID`	Ask / force a process to stop
`systemctl status nginx`	Is the service running?
`systemctl start|stop|restart nginx`	Control it
`systemctl enable nginx`	Start automatically at boot
`journalctl -u nginx -f`	Follow that service's logs

start a background process and stop it

```
$ sleep 300 &
$ ps -o pid,comm -p $!
$ kill $!
$ sleep 0.2
$ ps -p $! > /dev/null && echo "still running" || echo "stopped"
  PID COMMAND
 3613 sleep
stopped
```



A service lifecycle (systemd can auto-restart on failure)

PRO TIP

Debug flow for a broken service: `systemctl status` → `journalctl -u name -n 50` -> read the error → fix config → `systemctl restart`.

Packages, SSH and Transferring Files

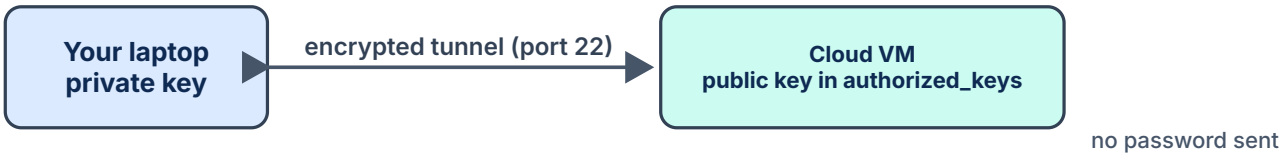
Installing software

package managers (illustrative, needs root)

```
# Ubuntu / Debian
$ sudo apt update && sudo apt install -y nginx
# Amazon Linux / RHEL / Fedora
$ sudo dnf install -y nginx
```

Always **update the package index first**; install only from trusted repositories.

SSH: secure remote login



Key-based login

create a key pair (illustrative)

```
$ ssh-keygen -t ed25519 -C "you@example.com"
$ ls ~/.ssh
# id_ed25519 = PRIVATE (keep secret), id_ed25519.pub = PUBLIC (share)
$ chmod 600 ~/.ssh/id_ed25519
```

connect and copy (illustrative)

```
$ ssh -i ~/.ssh/id_ed25519 ubuntu@203.0.113.25
$ scp report.pdf ubuntu@203.0.113.25:/home/ubuntu/
$ rsync -av ./site/ ubuntu@203.0.113.25:/var/www/site/
```

WATCH OUT

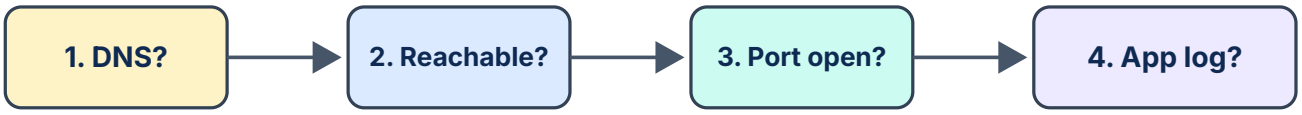
Share the **.pub** file, never the private key. Disable password login on servers once keys work.

Networking Commands and Logs

When something cannot connect, these commands narrow down *where* the problem is.

Command	Question it answers
`ping host`	Is the host reachable?
`curl -I https://example.com`	What does the web server answer? (headers only)
`dig example.com` / `nslookup`	Does DNS resolve the name?
`ss -tulpn`	Which ports are listening, and which program owns them?
`ip addr`	What are this machine's IPs?
`traceroute host`	Where along the path does it stop?

```
start a tiny web server and test it
$ python3 -m http.server 8765 --bind 127.0.0.1 >/dev/null 2>&1 &
$ sleep 1
$ curl -s -o /dev/null -w "HTTP status: %{http_code}\n" http://127.0.0.1:8765/
$ ss -tln | grep 8765 | awk '{print "listening on", $4}'
$ kill $!
---
HTTP status: 200
bash: line 7: ss: command not found
```



A troubleshooting ladder

Where logs live

`/var/log/syslog` or `/var/log/messages` (system), `/var/log/nginx/error.log` (web server), `journalctl` (systemd services). `tail -f file` follows a log live.

> **PRO TIP**
> Climb the ladder **bottom-up**: name, route, port, application. Most outages are found in the first two rungs.

Shell Scripting (1): Variables and Conditions

A **shell script** is a text file of commands. It turns a 12-step manual checklist into one repeatable command, the heart of DevOps.

check.sh

```
$ cat > check.sh <<'EOF'
#!/usr/bin/env bash
$ set -euo pipefail
$ name="${1:-world}"
$ echo "Hello, $name"
$ if [ -d /tmp ]; then
  echo "/tmp exists"
$ else
  echo "no /tmp"
$ fi
$ for n in 1 2 3; do
  echo "step $n"
$ done
$ EOF
$ chmod +x check.sh
$ ./check.sh Asha
```

```
Hello, Asha
/tmp exists
step 1
step 2
step 3
```

Piece	Meaning
`#!/usr/bin/env bash`	Which interpreter runs the file
`set -euo pipefail`	Stop on errors, undefined variables and failed pipes
`$1`, `$@`, `$#`	First argument, all arguments, argument count
`${1:-world}`	Default value if empty
`[ -f file ]`, `[ -d dir ]`	File / directory exists
`$?`	Exit code of the last command (0 = success)

PRO TIP

`set -euo pipefail` at the top is the single best habit for safer scripts. Without it, a script happily continues after something failed.

WATCH OUT

Quote your variables: `"$file"`, not `$file`. Unquoted names with spaces break commands.

Shell Scripting (2): A Backup Script and cron

backup.sh

```
$ mkdir -p data backups
$ echo "important" > data/a.txt
$ cat > backup.sh <<'EOF'
#!/usr/bin/env bash
$ set -euo pipefail
$ src="${1:?usage: backup.sh <folder>}"
$ stamp="$(date +%Y%m%d)"
$ out="backups/$(basename "$src")-$stamp.tar.gz"
$ tar -czf "$out" "$src"
$ echo "created $out"
$ EOF
$ chmod +x backup.sh
$ ./backup.sh data
$ tar -tzf backups/data-*.tar.gz
$ ./backup.sh || echo "exit code: $?"
```

```
created backups/data-20261111.tar.gz
data/
data/a.txt
exit code: 1
./backup.sh: line 3: 1: usage: backup.sh <folder>
```

Scheduling with cron

cron runs commands on a schedule. Edit your schedule with `crontab -e`. Five fields, then the command:

```
*/15 2 * * 1-5 /home/asha/backup.s
```

minute hour day month weekday command

Every 15 minutes during hour 2, Monday to Friday

Schedule	Meaning
`0 2 * * *`	Every day at 02:00
`*/5 * * * *`	Every 5 minutes
`30 18 * * 1-5`	18:30, Monday to Friday

WATCH OUT

cron has a tiny environment. Use **absolute paths** in cron jobs and redirect output to a log (`>> /var/log/backup.log 2>&1`).

Linux Cheat Sheet

Keep this page open while practising.

Navigate	
`pwd`	where am I
`ls -la`	list all
`cd ~` / `cd ..`	home / up
`find . -name x`	search
`tree`	folder tree

Files	
`cp -r a b`	copy
`mv a b`	move/rename
`rm -r d`	delete
`chmod 755 f`	permissions
`chown u:g f`	owner

Text	
`cat f`	show file
`less f`	page through
`grep -n x f`	search
`sort | uniq -c`	count
`tail -f log`	follow

System	
`ps aux`	processes
`top`	live usage
`df -h`	disk space
`free -h`	memory
`uname -a`	kernel info

Network	
`curl -I url`	headers
`ping h`	reachable
`ss -tulpn`	listening
`dig d`	DNS
`ip addr`	IPs

Services	
`systemctl status s`	state
`systemctl restart s`	restart
`journalctl -u s -f`	logs
`sudo !!`	rerun with sudo
`history | grep x`	find old cmd

three quick system checks

```
$ df -h / | tail -1 | awk '{print "disk used:", $5}'
$ nproc
$ uname -s
---
disk used: 29%
2
Linux
```

Git: Why Version Control?

Git tracks every change to your files, lets many people work in parallel and lets you go back in time. Cloud and DevOps work is mostly **text files in Git**: code, Dockerfiles, pipelines and infrastructure.

Without Git	With Git
`report_final_v2_REAL.docx`	One file, full history
"Who broke it?"	`git blame` shows who and when
Emailing zip files	Branches and pull requests
Fear of experiments	Experiment on a branch, discard freely

The three areas of Git



Files move through three areas

KEY IDEA

A **commit** is a snapshot with an ID, an author, a time and a message. A **branch** is just a movable label pointing at a commit. That is almost all of Git's model.

NOTE

Git (the tool) is not **GitHub** (a website that hosts Git repositories). GitLab and Bitbucket are alternatives.

Git Basics: Your First Repository

init, add, commit

```
$ git config user.name
$ mkdir hello-git && cd hello-git
$ git init
$ echo "# Hello Git" > README.md
$ git status -s
$ git add README.md
$ git commit -m "Add README"
$ git log --oneline
```

```
Asha Dev
Initialized empty Git repository in /home/user/hello-git/.git/
?? README.md
[main (root-commit) ca9036b] Add README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
ca9036b Add README
```

see what changed

```
$ echo "Practice notes" >> README.md
$ git diff
$ git commit -am "Update README"
```

```
diff --git a/README.md b/README.md
index ab690e8..ad37325 100644
--- a/README.md
+++ b/README.md
@@ -1 +1,2 @@
 # Hello Git
+Practice notes
[main 1117be7] Update README
 1 file changed, 1 insertion(+)
```

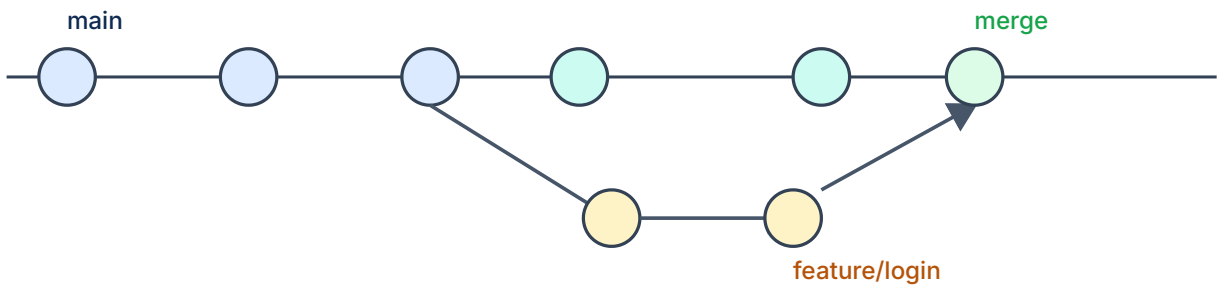
Command	Does
`git status`	What changed, what is staged
`git add f` / `git add .`	Stage a file / everything
`git commit -m "msg"`	Save a snapshot
`git diff`	Show unstaged changes

PRO TIP

Write commit messages in the imperative: *"Add login page"*, not *"added stuff"*. Future you reads them during outages.

Branching and Merging

A **branch** lets you work on a feature without touching `main`. When it is ready, you **merge** it back.



Feature branch forks from main and merges back

the feature-branch routine

```
$ git switch -c feature/login
$ echo "login page" > login.txt
$ git add login.txt
$ git commit -m "Add login page"
$ git switch main
$ git merge feature/login
$ git log --oneline --graph
$ git branch -d feature/login
```

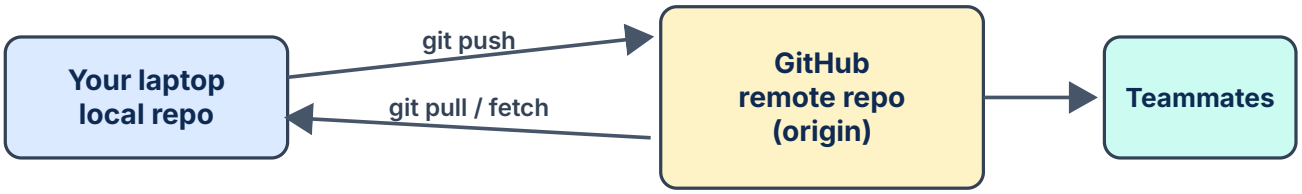
```
[feature/login a26b34f] Add login page
 1 file changed, 1 insertion(+)
 create mode 100644 login.txt
Updating 045444f..a26b34f
Fast-forward
 login.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 login.txt
* a26b34f Add login page
* 045444f Initial commit
Deleted branch feature/login (was a26b34f).
Switched to a new branch 'feature/login'
Switched to branch 'main'
```

PRO TIP

Create a branch for **every change**. Short-lived branches merged often are much easier than long-lived ones.

Remotes, GitHub and Pull Requests

A **remote** is a copy of your repository on a server (GitHub). `origin` is the default name for it.



Push shares, pull receives

```
clone, commit and push (local 'remote' stands in for GitHub)
$ git clone remote.git copy > /dev/null 2>&1
$ cd copy
$ echo "change" >> app.txt
$ git commit -qam "Edit app"
$ git push -q origin main
$ git log --oneline origin/main
$ git remote -v
---
a496d0b Edit app
045444f Initial commit
origin  /home/user/remote.git (fetch)
origin  /home/user/remote.git (push)
```

1. Create a branch and commit your work.
2. `git push -u origin feature/x`.
3. Open a **Pull Request** on GitHub: describe what and why.
4. Teammates review; automated checks run (Chapter 5).
5. Merge, delete the branch, `git pull` on main.

WATCH OUT

Connect to GitHub with **SSH keys** or a credential helper. Do not paste passwords or tokens into commands or scripts.

Ignoring Files, Stash and Undoing Mistakes

.gitignore

List files Git must **never track**: secrets, build output, dependencies.

a typical .gitignore

```
.env
*.pem
node_modules/
__pycache__/
dist/
*.log
```

Undo cheat sheet

Situation	Command
Discard edits to a file	`git restore file`
Unstage a file	`git restore --staged file`
Fix the last commit message	`git commit --amend`
Undo a pushed commit safely	`git revert <id>` (adds a new commit)
Park unfinished work	`git stash` then `git stash pop`
Move branch pointer back (local only)	`git reset --hard <id>` (destroys work!)

restore and stash

```
$ echo "oops" >> app.txt
$ git restore app.txt
$ git status -s
$ echo "wip" > wip.txt && git add wip.txt
$ git stash -q
$ git status -s
$ git stash pop -q
$ git status -s
---------------------------------------------------------------------------------------------------
A  wip.txt
```

WATCH OUT

If a secret (password, key) was ever committed, **rotate it**. Deleting it in a later commit does not remove it from history.

Merge Conflicts and Rebase

A **conflict** happens when two branches change the same line. Git cannot choose, so it marks the file and waits for you.

```
create a conflict on purpose
$ git switch -q -c fix
$ echo "color=blue" > config.txt
$ git commit -qam "blue"
$ git switch -q main
$ echo "color=red" > config.txt
$ git commit -qam "red"
$ git merge fix || true
$ cat config.txt
---
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
<<<<<<< HEAD
color=red
=======
color=blue
>>>>>>> fix
```

1. Open the file; keep the final text and delete the `<<<<<<<`, `=======`, `>>>>>>>` markers.
2. `git add config.txt`
3. `git commit` to finish the merge.

Merge vs rebase

	Merge	Rebase
History	Keeps true history, adds a merge commit	Replays your commits on top of main: straight line
Use when	Combining shared branches	Cleaning your *own* branch before a PR
Rule	Safe everywhere	Never rebase commits others already pulled

PRO TIP

Conflicts are normal, not failures. Small branches and frequent `git pull` keep them small.

Git Workflows, Recap and Lab

Common team workflows

Workflow	How it works	Good for
GitHub flow	`main` is always deployable; short feature branches + pull requests	Most web teams, CI/CD friendly
Trunk-based	Everyone merges to main daily behind feature flags	High-frequency delivery
Git flow	Long-lived `develop`, `release`, `hotfix` branches	Versioned releases

Chapter recap

- ☐ Linux: tree filesystem, relative vs absolute paths
- ☐ `chmod`, `sudo`, least privilege
- ☐ Pipes and redirects build powerful one-liners
- ☐ Scripts start with `set -euo pipefail`; cron schedules them
- ☐ Git: working directory > staging > repository
- ☐ Branch for every change; merge through pull requests
- ☐ Never commit secrets; rotate if you did

⚒ Hands-on lab: A script in a repository

1. Make a folder `devops-lab` and `git init` it.
2. Write `backup.sh` from this chapter and commit it on `main`.
3. Create branch `feature/log` and make the script append a line to `backup.log`.
4. Add `*.log` and `backups/` to `.gitignore`.
5. Push to a new GitHub repository and open a pull request on your own repo.

Why Containers?

CHAPTER 4

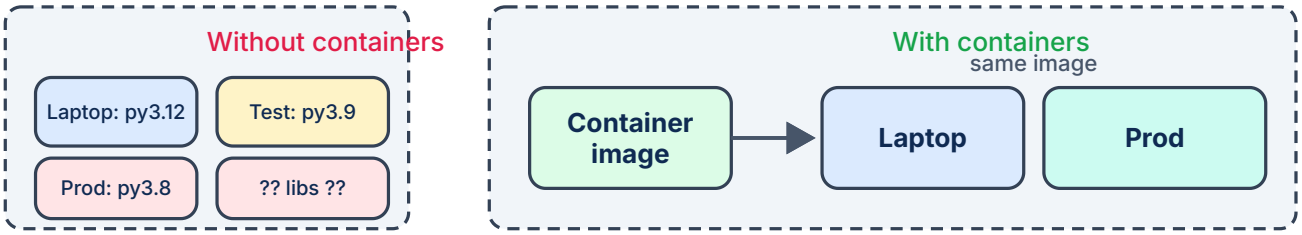
04 Docker and Containerization

Package once, run anywhere

WHAT YOU WILL LEARN

- Why containers solved the 'works on my machine' problem
- Images, layers, Dockerfiles and multi-stage builds
- Volumes, networks and Docker Compose
- Registries, security habits and a first look at Kubernetes

Every developer has said it: *"but it works on my machine!"* The app needs Python 3.12, a certain library version and an environment variable, and the server has none of them. A **container** bundles the app **and everything it needs** into one portable unit.



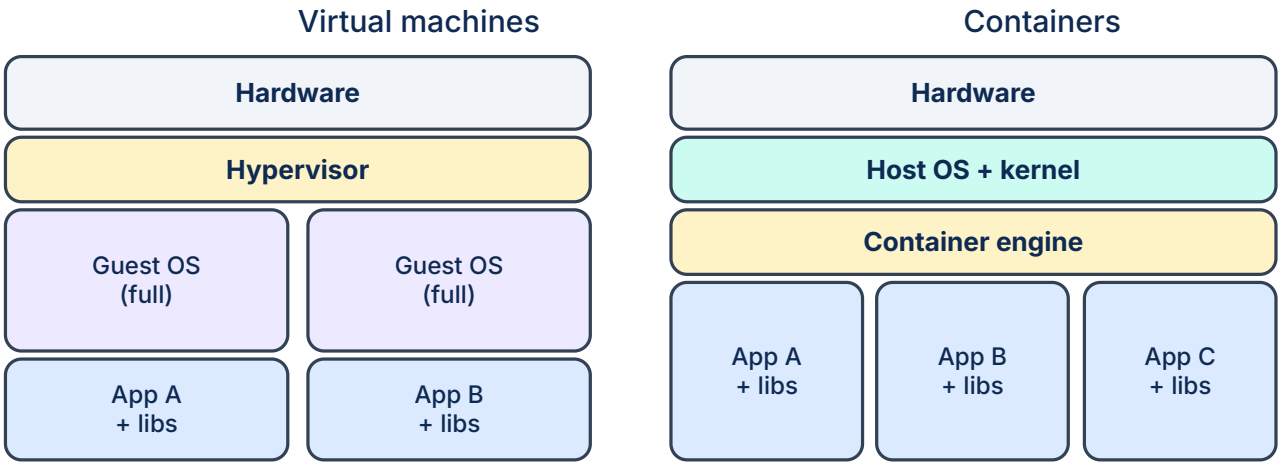
One image behaves the same everywhere

Benefit	Why it matters
Consistency	Dev, test and prod run the identical package
Speed	Starts in seconds, not minutes
Density	Many containers share one machine's resources
Isolation	Apps cannot see each other's files or processes by default
Automation	Perfect unit for CI/CD pipelines

KEY IDEA

A container is **not** a tiny VM. It is a normal Linux process with a restricted view of the system.

Containers vs Virtual Machines



Containers share the host kernel; VMs carry a whole OS each

	Virtual machine	Container
Size	Gigabytes	Megabytes
Start time	Minutes	Seconds or less
Isolation	Strong (own kernel)	Process-level (shared kernel)
OS	Any guest OS	Same kernel family as host
Density	Tens per server	Hundreds per server
Best for	Different OSes, strong isolation	Microservices, CI jobs, fast scaling

How containers isolate things (Linux features)

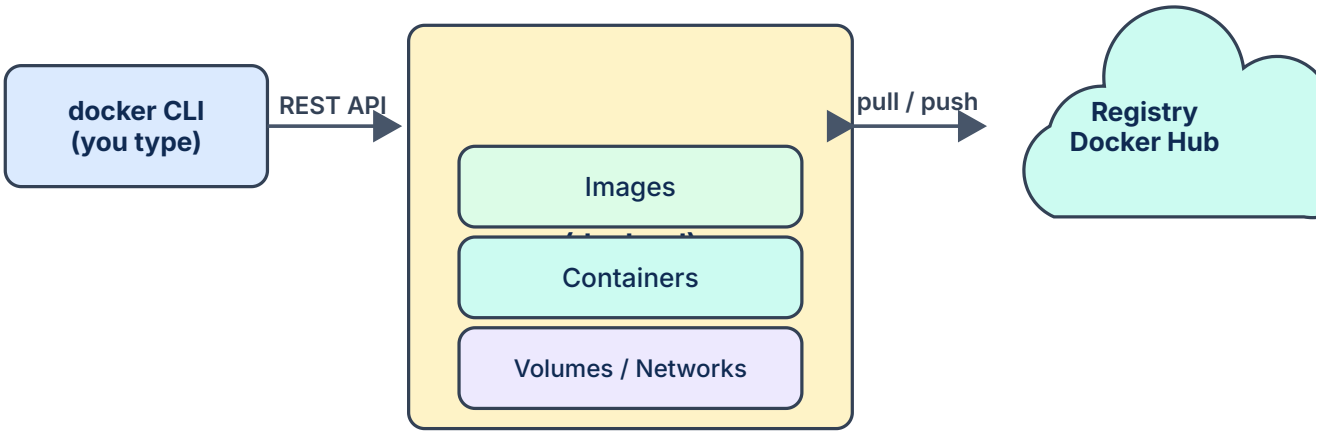
- **Namespaces**: each container sees its own processes, network and filesystem view
- **cgroups**: limits on CPU and memory per container
- **Union filesystems**: images built from shared read-only layers

PRO TIP

In practice, teams often run containers **inside** VMs: the VM gives strong boundaries, containers give speed and portability.

Docker Architecture

Docker is the most popular tool for building and running containers. Four pieces work together:



Client, daemon and registry

Term	Meaning
Image	A read-only template: app + dependencies + metadata
Container	A running (or stopped) instance of an image
Dockerfile	A text recipe for building an image
Registry	A server that stores images (Docker Hub, ECR, ACR, GAR)
Docker Compose	Tool to run multi-container apps from one YAML file

KEY IDEA

Image : container as **class : object**, or **recipe : cooked meal**. One image can start many containers.

NOTE

Docker is the best-known tool but containers follow open standards (OCI). Podman, containerd and Kubernetes run the same images.

Install and First Commands

Install **Docker Desktop** (Windows/macOS) or Docker Engine (Linux), then verify:

```
your first containers (illustrative; needs Docker running)
$ docker --version
$ docker run hello-world
$ docker run -d --name web -p 8080:80 nginx
$ docker ps
$ curl http://localhost:8080
$ docker logs web
$ docker exec -it web bash
$ docker stop web
$ docker rm web
```

Command	Does
`docker run image`	Create and start a container
`-d`	Detached (background)
`-p 8080:80`	Publish container port 80 on host port 8080
`--name web`	Give it a name
`-it`	Interactive terminal
`--rm`	Delete the container when it exits
`-e KEY=value`	Set an environment variable
`docker ps` / `ps -a`	Running / all containers
`docker logs -f web`	Follow output
`docker exec -it web sh`	Open a shell inside

PRO TIP

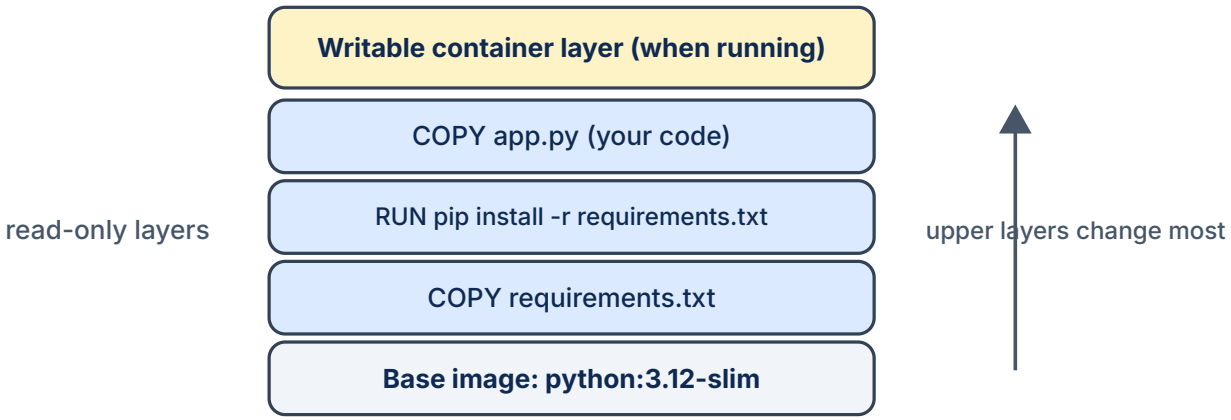
On Linux add yourself to the `docker` group (`sudo usermod -aG docker $USER`) and log in again, so you do not need `sudo` for every command.

WATCH OUT

Anyone in the `docker` group effectively has root on that machine. Do not add untrusted users.

Images and Layers

An image is a **stack of read-only layers**. Each Dockerfile instruction adds a layer. Layers are **cached and shared**, which is why rebuilds and downloads are fast.



Layers stack; only the top writable layer changes at runtime

```
inspecting images (illustrative)
$ docker pull python:3.12-slim
$ docker images
$ docker history python:3.12-slim
$ docker rmi python:3.12-slim
```

Image names and tags

```
anatomy
registry/namespace/name:tag
docker.io/library/python:3.12-slim
ghcr.io/asha/shop-api:1.4.2
```

WATCH OUT

Avoid the `latest` tag in production. It moves. Use a **specific version** (`3.12-slim`, `1.4.2`) so builds are repeatable.

KEY IDEA

Cache rule: put things that change rarely (dependency install) *before* things that change often (your source code).

Writing a Dockerfile

A **Dockerfile** is a recipe. `docker build` follows it top to bottom, one layer per instruction.

Instruction	Purpose
`FROM`	Base image to start from
`WORKDIR`	Set the working directory
`COPY` / `ADD`	Copy files into the image (prefer `COPY`)
`RUN`	Execute a command at **build** time (install packages)
`ENV`	Set environment variables
`EXPOSE`	Document the port the app uses
`USER`	Run as a non-root user
`CMD`	Default command at **run** time
`ENTRYPOINT`	Fixed executable; `CMD` becomes its arguments

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

build and run

```
$ docker build -t shop-api:1.0 .
$ docker run --rm -p 5000:5000 shop-api:1.0
```

KEY IDEA

`RUN` happens while **building**; `CMD` happens when the container **starts**. Mixing them up is the most common beginner error.

PRO TIP

Use the **exec form** `CMD ["python","app.py"]` (a JSON list) so signals reach your app and it can stop cleanly.

Dockerfile Example: A Flask API

The app below is tiny but real. It was tested with Flask's test client before printing.

app.py

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.get("/")
def home():
    return jsonify(message="Hello from a container!")

@app.get("/health")
def health():
    return jsonify(status="ok")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

```
{'message': 'Hello from a container!'}
200 {'status': 'ok'}
```

requirements.txt

```
flask==3.1.3
gunicorn==23.0.0
```

Dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
RUN useradd --create-home appuser
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

WATCH OUT

`app.run()` is a development server. Containers use **gunicorn**, bound to `0.0.0.0`. Add a `.dockerignore` (`.git/`, `.env`, `__pycache__/`) so junk never enters the image.

Dockerfile Example: A Node.js App

The same pattern works for any language. Only the base image and install command change.

server.js

```
const http = require("http");
const port = process.env.PORT || 3000;

http.createServer((req, res) => {
  res.writeHead(200, {"Content-Type": "application/json"});
  res.end(JSON.stringify({ message: "Hello from Node in a container" }));
}).listen(port, () => console.log("listening on " + port));
```

Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV PORT=3000
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

Language	Base image idea	Install step
Python	`python:3.12-slim`	`pip install -r requirements.txt`
Node.js	`node:22-alpine`	`npm ci --omit=dev`
Java	`eclipse-temurin:21-jre`	copy the built `.jar`
Go	`golang` to build, `scratch` /distroless to run	`go build`

PRO TIP

slim and **alpine** images are much smaller than the full ones, so they download faster and expose fewer vulnerabilities. Alpine uses a different C library, so test your app on it.

NOTE

Use `npm ci` (not `npm install`) in builds: it installs exactly what `package-lock.json` says.

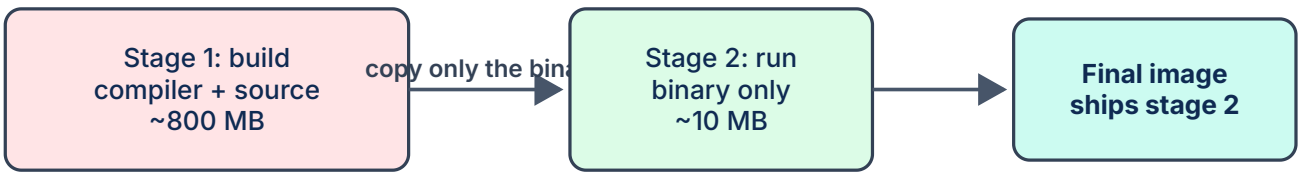
Multi-Stage Builds

Build tools (compilers, test libraries) are needed to **build** an app but not to **run** it. A **multi-stage build** uses one stage to build and a second, tiny stage to ship.

Dockerfile (Go example)

```
# ---- stage 1: build ----
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /out/server .

# ---- stage 2: run ----
FROM gcr.io/distroless/static-debian12
COPY --from=build /out/server /server
USER nonroot
EXPOSE 8080
ENTRYPOINT ["/server"]
```



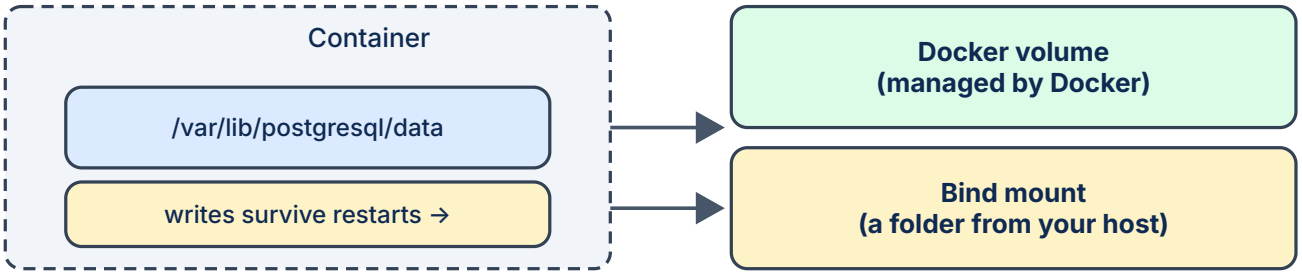
Only the last stage is shipped

KEY IDEA

Smaller images mean faster deployments, cheaper registry storage and a smaller attack surface. Multi-stage builds are a standard interview topic.

Data: Volumes and Bind Mounts

A container's own filesystem is **temporary**. When the container is removed, anything written inside is gone. To keep data, mount storage from outside.



Volumes live outside the container

Type	Syntax	Best for
Named volume	`-v pgdata:/var/lib/postgresql/data`	Databases and production data
Bind mount	`-v $(pwd):/app`	Live code editing during development
tmpfs	`--tmpfs /tmp`	Temporary in-memory data

```
data outlives the container (illustrative)
$ docker volume create pgdata
$ docker run -d --name db -e POSTGRES_PASSWORD=devonly \
  -v pgdata:/var/lib/postgresql/data postgres:16
$ docker rm -f db
$ docker run -d --name db2 -e POSTGRES_PASSWORD=devonly \
  -v pgdata:/var/lib/postgresql/data postgres:16   # data is still there
$ docker volume ls
```

KEY IDEA

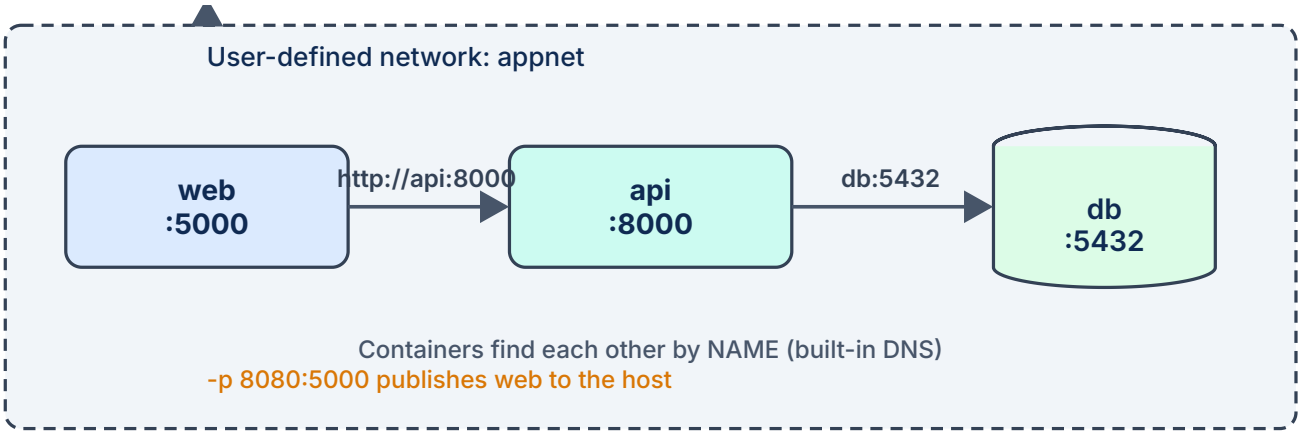
Containers should be **disposable** (cattle, not pets). Keep state in volumes or managed services, and keep the container itself replaceable.

WATCH OUT

Do not store secrets in images or bind-mounted `.env` files that get committed.

Docker Networking

Containers talk to each other and the world through **Docker networks**.



Names, not IP addresses

Driver	Behaviour
bridge (default)	Private network on one host; use a user-defined bridge for name lookup
host	Container shares the host network (Linux)
none	No networking
overlay	Multi-host networking (Swarm / orchestrators)

```
create a network (illustrative)
$ docker network create appnet
$ docker run -d --name api --network appnet my-api:1.0
$ docker run -d --name web --network appnet -p 8080:5000 my-web:1.0
$ docker network inspect appnet
```

PRO TIP

Only **publish** (`-p`) the ports that the outside world needs. A database should be reachable by other containers on the network but not published to the host.

WATCH OUT

Inside a container, `localhost` means **that container**, not your laptop. Use the service name instead (`db` , `redis`).

Docker Compose (1): Running Several Containers

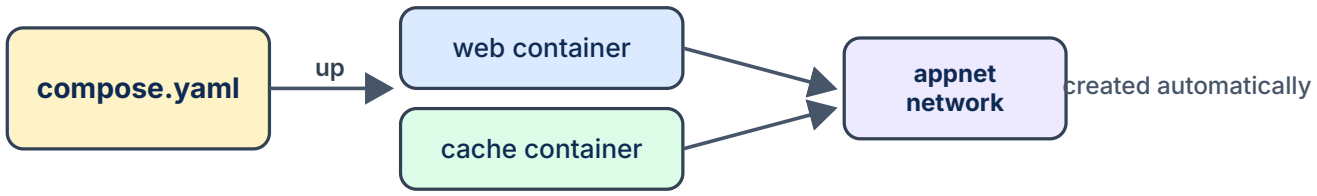
Real apps are many containers. **Docker Compose** describes them in one `compose.yaml` file and starts everything with a single command.

compose.yaml

```yaml
services:
  web:
    build: .
    ports:
      - "8080:5000"
    environment:
      REDIS_HOST: cache
    depends_on:
      - cache
  cache:
    image: redis:7-alpine
```

daily commands (illustrative)

```
$ docker compose up -d --build
$ docker compose ps
$ docker compose logs -f web
$ docker compose down
```



Compose creates containers and a shared network

NOTE

`docker compose up` (v2) replaces the old `docker-compose up`. Compose files are plain YAML: **indentation uses spaces, never tabs**.

WATCH OUT

`depends_on` only controls **start order**, not that the service is *ready*. Use a `healthcheck` for that (next page).

Docker Compose (2): A Full Stack

compose.yaml

```yaml
services:
  web:
    build: .
    ports:
      - "8080:5000"
    environment:
      DATABASE_URL: postgresql://app:${DB_PASSWORD}@db:5432/shop
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: shop
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d shop"]
      interval: 5s
      timeout: 3s
      retries: 5
volumes:
  pgdata:
```

.env (next to compose.yaml, listed in .gitignore)

```
DB_PASSWORD=change-me-locally
```

Key	Meaning
`build`	Build image from a Dockerfile
`image`	Use a prebuilt image
`ports`	Host:container port publish
`healthcheck`	Command that proves the service is ready

WATCH OUT

`${DB_PASSWORD}` is read from the `.env` file or your shell. **Never commit** `.env`; commit a `.env.example` instead.

Registries and Tagging

A **registry** stores images so other machines (servers, pipelines, teammates) can pull them.

Registry	Where	Notes
Docker Hub	docker.io	Default public registry
GitHub Container Registry	ghcr.io	Great with GitHub Actions
Amazon ECR	`<acct>.dkr.ecr.<region>.amazonaws.com`	Private, integrates with IAM
Azure Container Registry	`<name>.azurecr.io`	Private, integrates with Azure RBAC
Google Artifact Registry	`<region>-docker.pkg.dev`	Private, integrates with Cloud IAM

```
build, tag, push (illustrative)
$ docker login ghcr.io
$ docker build -t ghcr.io/asha/shop-api:1.4.2 .
$ docker tag ghcr.io/asha/shop-api:1.4.2 ghcr.io/asha/shop-api:latest
$ docker push ghcr.io/asha/shop-api:1.4.2
$ docker pull ghcr.io/asha/shop-api:1.4.2
```



Registry sits between build and run

A good tagging scheme

- **Version tag**: `1.4.2` (immutable, human friendly)
- **Git commit tag**: `sha-3f9a2c1` (exactly traceable)
- **Moving tag**: `latest` or `main` for convenience only

PRO TIP

Scan images for known vulnerabilities in the registry or CI (Trivy, Docker Scout, cloud-native scanners).

Container Security and Troubleshooting

Security checklist

- ☐ Start from small, trusted base images; pin versions
- ☐ Run as a **non-root** user (`USER appuser`)
- ☐ Keep secrets out of images; inject at runtime
- ☐ Scan images for vulnerabilities in CI
- ☐ Use read-only filesystems where possible: `--read-only`
- ☐ Set CPU and memory limits: `--memory 512m --cpus 1`
- ☐ Rebuild images regularly to pick up security fixes

Troubleshooting cheat sheet

Symptom	First commands	Common cause
Container exits immediately	`docker logs NAME`, `docker ps -a`	App crashed, wrong `CMD`, missing env var
Cannot reach app on localhost	`docker port NAME`, `docker ps`	Forgot `-p`, or app bound to `127.0.0.1`
Port already allocated	`ss -tulpn | grep 8080`	Another process uses the host port
Build is slow	`docker build` output	Cache busted by `COPY . .` too early
Image huge	`docker history IMAGE`	No multi-stage build, big base image
Cannot connect between containers	`docker network inspect`	Different networks, using `localhost`
Disk full	`docker system df`, `docker system prune`	Old images, volumes, build cache

PRO TIP

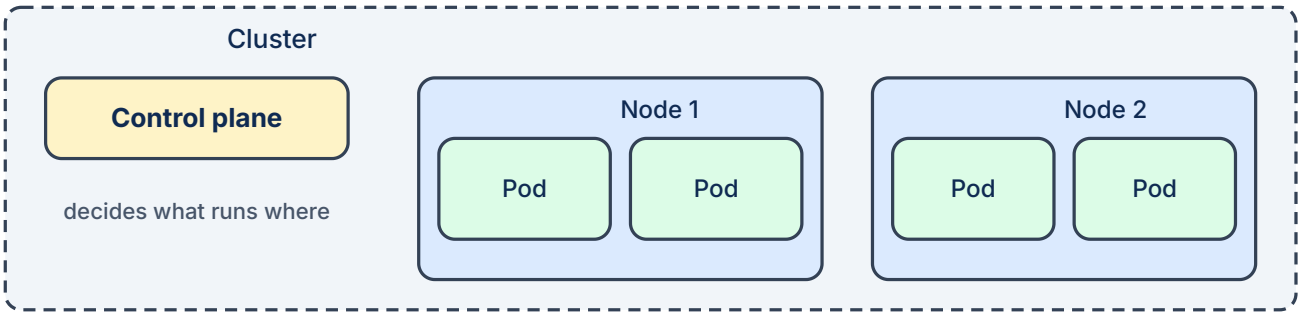
`docker exec -it NAME sh` drops you inside a running container to look around. Reading logs first solves most problems.

WATCH OUT

`docker system prune -a` deletes unused images; add `--volumes` and it can delete data. Read the confirmation prompt.

Kubernetes Intro, Recap and Lab

Running a few containers is easy. Running **hundreds across many servers** needs an **orchestrator**. **Kubernetes (K8s)** is the standard one: it schedules containers, restarts failures, scales them and rolls out updates.



Pods run on nodes; the control plane decides placement

Key objects: **Pod** (one or more containers), **Deployment** (keeps N copies running, rolling updates), **Service** (stable address and load balancing), **Ingress** (HTTP routing).

> **NOTE**
> Managed Kubernetes services: **EKS** (AWS), **AKS** (Azure), **GKE** (Google). Learn Docker first; Kubernetes is the natural next step.

Chapter recap

- ☐ Containers share the host kernel; images are layered
- ☐ Dockerfile: `FROM`, `COPY`, `RUN`, `CMD`, non-root `USER`
- ☐ Multi-stage builds shrink images
- ☐ Volumes keep data; networks connect by name
- ☐ Compose runs multi-container apps from YAML
- ☐ Never trust `latest`; scan images

⚒ Hands-on lab: Containerise a Flask app

1. Save `app.py` and `requirements.txt` from this chapter.
2. Write the Dockerfile, build, and run with `-p 5000:5000`; visit `/health`.
3. Add a `.dockerignore` and compare image size before and after.
4. Create a `compose.yaml` with the app and Redis; run `docker compose up`.
5. Push the image to a registry with a version tag.

What Is CI/CD?

05 CHAPTER 5 CI/CD Fundamentals

Ship small changes safely, many times a day

WHAT YOU WILL LEARN

- Continuous Integration, Delivery and Deployment
- Pipeline stages, automated tests and GitHub Actions
- Deployment strategies, IaC and secrets handling
- Monitoring, SLOs and DORA metrics

Without automation, releasing software is a nervous ritual: merge for weeks, test by hand, deploy on Friday night, hope. **CI/CD** replaces the ritual with a **pipeline**, an automated assembly line that checks every change.



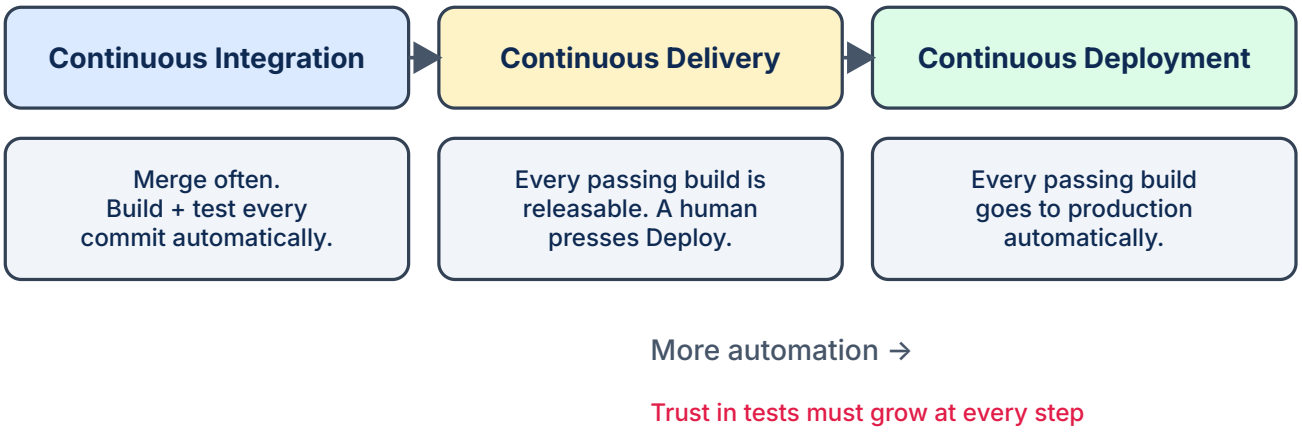
Every commit travels down the same automated line

Without CI/CD	With CI/CD
Big, rare, scary releases	Small, frequent, boring releases
Bugs found by customers	Bugs found by the pipeline in minutes
"Works on my machine"	Same automated checks for everyone
Manual copy-paste deploys	One-click or automatic deploys

KEY IDEA

The goal is **fast feedback**: tell the developer within minutes whether a change is safe, while they still remember what they wrote.

CI vs Continuous Delivery vs Deployment



Three levels of automation

A typical pipeline, stage by stage

Stage	What happens	Fails when
Source	Pipeline starts on push or pull request	N/A
Build	Install dependencies, compile or build image	Code does not compile
Test	Lint, unit tests, integration tests	A test fails
Scan	Dependency and image vulnerability scans	Critical vulnerability
Package	Create artifact or Docker image, tag with version	Registry unreachable
Deploy	Release to staging, then production	Health check fails
Verify	Smoke tests and monitoring	Error rate rises

PRO TIP

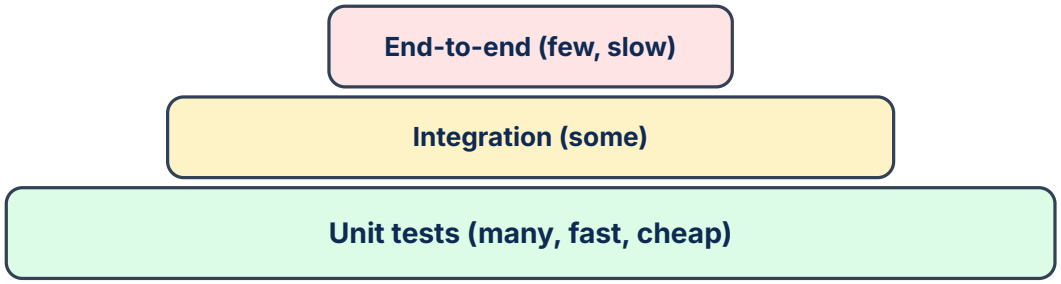
Fail fast: put quick checks (lint, unit tests) first. Do not spend ten minutes building an image for code that has a syntax error.

WATCH OUT

A pipeline is only as good as its tests. Green does not mean *correct*, it means *the checks you wrote passed*.

Automated Testing with pytest

Tests are the safety net of CI. The **testing pyramid** says: many fast unit tests, fewer integration tests, very few slow end-to-end tests.



The testing pyramid

test_pricing.py

```
def apply_discount(price, percent):
    if not 0 ⩽ percent ⩽ 100:
        raise ValueError("percent must be between 0 and 100")
    return round(price * (1 - percent / 100), 2)

import pytest

def test_normal_discount():
    assert apply_discount(200, 10) == 180.0

def test_zero_discount():
    assert apply_discount(99.99, 0) == 99.99

def test_invalid_percent():
    with pytest.raises(ValueError):
        apply_discount(100, 150)
```

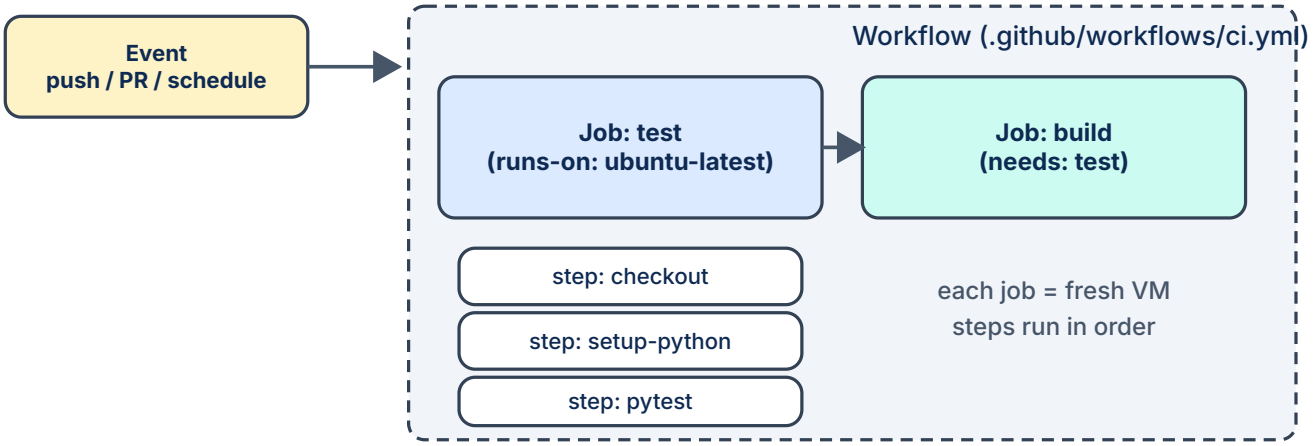
```
...                                                    [100%]
3 passed in 0.00s
```

PRO TIP

Test the **boring edge cases**: empty input, zero, negative numbers, missing fields. Most production bugs live there.

GitHub Actions: Concepts

GitHub Actions runs workflows defined in YAML files stored in your repository under `.github/workflows/`. No separate server needed.



Event > workflow > jobs > steps

Term	Meaning
Workflow	One YAML file describing an automation
Event	What triggers it: `push`, `pull_request`, `schedule`, `workflow_dispatch`
Job	A group of steps on one runner; jobs run in parallel unless `needs:` is set
Step	One command (`run:`) or reusable action (`uses:`)
Runner	The machine that executes a job (GitHub-hosted or self-hosted)
Action	A reusable packaged step, e.g. `actions/checkout`

WATCH OUT

Each job starts on a **clean machine**. Nothing carries over between jobs unless you pass **artifacts** or use a cache.

GitHub Actions: A Python CI Workflow

.github/workflows/ci.yml

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: Install dependencies
        run: pip install -r requirements.txt pytest

      - name: Run tests
        run: pytest -q
```

1. `on:` makes the workflow run on pushes to `main` and on every pull request.
2. `checkout` downloads your repository onto the fresh runner.
3. `setup-python` installs Python and caches pip downloads.
4. `run:` executes shell commands; any non-zero exit **fails** the job.

TRY IT

Commit the file, push, then open the **Actions** tab on GitHub to watch it run. A red cross on a pull request blocks merging when you enable **branch protection**.

PRO TIP

Action versions such as `@v4` are tags that can move. For third-party actions, **pin to a full commit SHA** for supply-chain safety.

GitHub Actions: Matrix, Caching and Artifacts

test on three Python versions

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.10", "3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: pip
      - run: pip install -r requirements.txt pytest
      - run: pytest -q --junitxml=report.xml
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: test-report-${{ matrix.python-version }}
          path: report.xml
```

Feature	Why use it
Matrix	Run the same job across versions or operating systems in parallel
Cache	Reuse downloaded dependencies to speed up runs
Artifacts	Keep files (reports, builds) after the job ends
`if: always()`	Run a step even if earlier steps failed
`needs:`	Make one job wait for another

KEY IDEA

`${{ ... }}` is the expression syntax: it reads values such as `matrix.python-version`, `github.sha` or `secrets.NAME`.

GitHub Actions: Build and Push a Docker Image

Add a second job that builds the image from Chapter 4 and pushes it to GitHub Container Registry, only after tests pass and only on `main`.

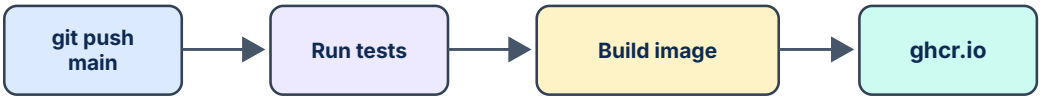
.github/workflows/docker.yml

```yaml
name: Build and push

on:
  push:
    branches: [main]

permissions:
  contents: read
  packages: write

jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ghcr.io/${{ github.repository }}:latest
            ghcr.io/${{ github.repository }}:${{ github.sha }}
```



Commit to registry, automatically

WATCH OUT

`secrets.GITHUB_TOKEN` is created for each run and needs no setup. For other registries or clouds, store credentials in **Settings > Secrets and variables > Actions**, never in the YAML.

Other CI Tools: Jenkins and GitLab CI

The ideas are identical in every CI system. Only the file format changes.

Jenkinsfile (declarative pipeline)

```
pipeline {
  agent any
  stages {
    stage('Install') { steps { sh 'pip install -r requirements.txt pytest' } }
    stage('Test')    { steps { sh 'pytest -q' } }
    stage('Build')   { steps { sh 'docker build -t shop-api:${BUILD_NUMBER} .' }
}
  }
  post { failure { echo 'Build failed!' } }
}
```

.gitlab-ci.yml

```
stages: [test, build]

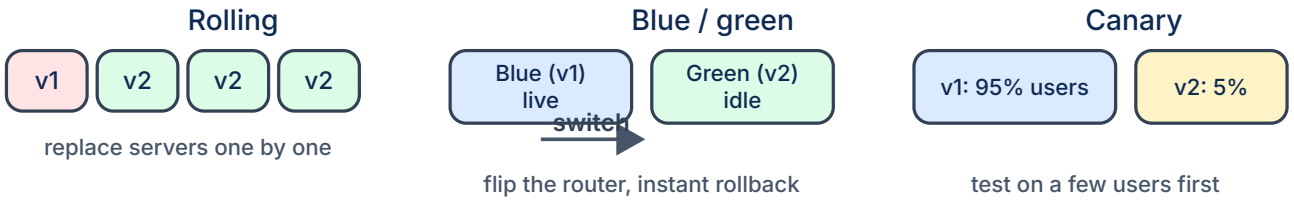
test:
  stage: test
  image: python:3.12-slim
  script:
    - pip install -r requirements.txt pytest
    - pytest -q

build:
  stage: build
  image: docker:27
  services: [docker:27-dind]
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA .
  only: [main]
```

Tool	Hosting	Config file	Strength
GitHub Actions	Cloud (GitHub)	`.github/workflows/*.yml`	Tight GitHub integration, huge action marketplace
GitLab CI	Cloud or self-hosted	`.gitlab-ci.yml`	All-in-one DevOps platform
Jenkins	Self-hosted	`Jenkinsfile`	Extremely flexible, many plugins
Cloud-native	AWS CodePipeline, Azure DevOps, Cloud Build	varies	Close to the cloud's IAM and services

Deployment Strategies

How do you replace the old version with the new one without users noticing?



Three ways to release

Strategy	How it works	Pros	Cons
Rolling	Replace instances gradually	No extra capacity	Two versions live briefly; slower rollback
Blue / green	Run full new copy, then switch traffic	Instant rollback, easy testing	Needs double capacity for a while
Canary	Send a small % of traffic to the new version, watch metrics, increase	Limits damage of a bad release	Needs good monitoring and traffic control
Recreate	Stop old, start new	Simple	Downtime

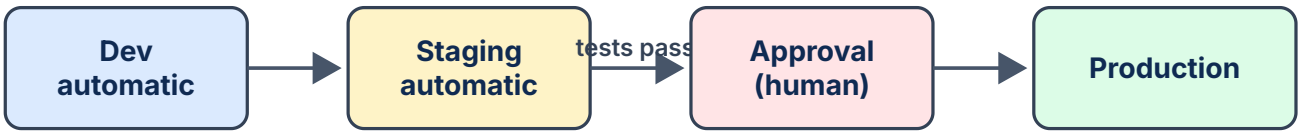
Database changes: the tricky part

- Make schema changes **backward compatible** (add columns before using them).
- Deploy code that works with **both** old and new schema, then remove the old path later.
- Always take a backup before a migration.

PRO TIP

Pick the strategy that matches your risk and budget. Most teams start with rolling updates and add canary releases later.

Environments, Approvals and Rollback



Promote the same build through environments

Build once, deploy many: create the artifact (image) a single time and promote that exact artifact. Only configuration changes between environments.

Environment	Purpose	Data
Development	Quick experiments	Fake
Staging / test	Production-like rehearsal	Anonymised or synthetic
Production	Real users	Real, protected

GitHub environment with approval

```
jobs:
  deploy-prod:
    runs-on: ubuntu-latest
    needs: deploy-staging
    environment: production   # requires reviewers set in repo settings
    steps:
      - run: echo "deploying ${{ github.sha }} to production"
```

Rollback plan

- ☐ Keep the previous image tag so you can redeploy it in minutes
- ☐ Make migrations reversible or additive
- ☐ Use **feature flags** to switch a risky feature off without a deploy
- ☐ Write down who decides to roll back and how

KEY IDEA

The fastest fix for a bad release is usually a **rollback**, not a hurried patch.

Infrastructure as Code: Terraform and Ansible

Clicking in a console is slow and impossible to review. **Infrastructure as Code (IaC)** describes servers and networks in text files kept in Git, so infrastructure gets the same review, history and automation as software.

Tool	Style	Typical use
Terraform / OpenTofu	Declarative, multi-cloud	Create cloud resources (VPCs, VMs, buckets)
CloudFormation / Bicep	Declarative, one cloud	AWS / Azure native
Ansible	Procedural, agentless (SSH)	Configure software on servers
Pulumi / CDK	Real programming languages	Infrastructure from Python / TypeScript

main.tf

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "notes" {
  bucket = "asha-notes-bucket-2026"
  tags   = { Project = "devops-lab" }
}
```

the four commands (illustrative; needs a cloud account)

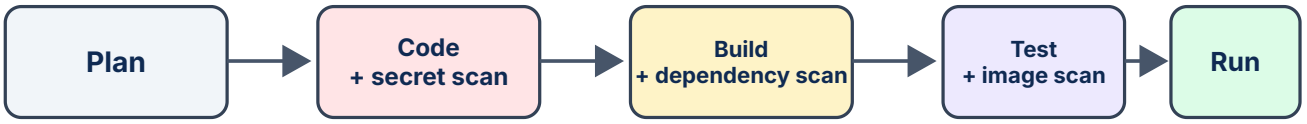
```
$ terraform init      # download providers
$ terraform plan      # preview changes, touches nothing
$ terraform apply     # create the resources
$ terraform destroy   # remove everything it created
```

WATCH OUT

The **plan** step is the safety net: read it before every apply. Store Terraform *state* in a remote backend (such as an object-storage bucket with locking), not on your laptop.

Secrets and DevSecOps

A **secret** is any value that grants access: passwords, API keys, tokens, private keys. **DevSecOps** means building security checks into the pipeline, not bolting them on at the end.



Shift security left

Do	Do not
Store secrets in a manager: GitHub Secrets, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager	Hard-code keys in code, Dockerfiles or YAML
Use short-lived credentials and OIDC from CI to the cloud	Create permanent admin keys for pipelines
Scan dependencies and images (Dependabot, Trivy)	Ignore vulnerability reports
Add a **secret scanner** (gitleaks, GitHub secret scanning)	Rely on people noticing a leaked key
Give each pipeline least-privilege access	Run everything as root/admin

WATCH OUT

If a secret leaks: **revoke and rotate first**, investigate second. Removing it from the latest commit does not remove it from Git history.

PRO TIP

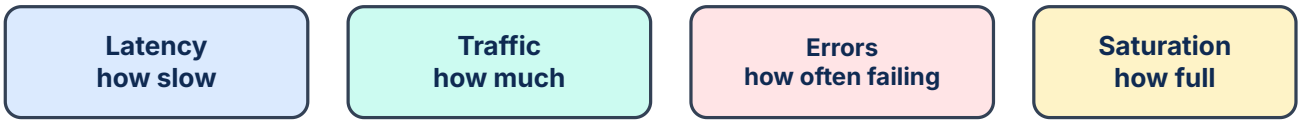
OIDC lets GitHub Actions obtain **temporary** cloud credentials without storing any long-lived key. It is the modern best practice for deploying to AWS, Azure or Google Cloud.

Monitoring, Observability and SLOs

Deploying is half the job. After release you must know whether the system is healthy.

Signal	Question	Example
Metrics	How much? How fast?	Requests per second, error rate, CPU
Logs	What happened?	`ERROR payment failed order=88`
Traces	Where was the time spent?	API > database took 800 ms

The four golden signals



What to watch for any service

Term	Meaning	Example
SLI	A measured indicator	% of requests answered under 300 ms
SLO	A target for the SLI	99.9% over 30 days
SLA	A contract with penalties	Refund if below 99.5%
Error budget	Allowed failure = 100% - SLO	0.1% means about 43 minutes a month

error_budget.py

```
slo = 99.9
minutes_30_days = 30 * 24 * 60
print("error budget:", round(minutes_30_days * (100 - slo) / 100, 1), "minutes")
```

```
error budget: 43.2 minutes
```

KEY IDEA

If the error budget is spent, **slow down releases** and fix reliability. If plenty is left, ship faster. SLOs turn arguments about risk into numbers.

DORA Metrics, Recap and Lab

How do you know your pipeline is working? The **DORA** research programme found four metrics that separate high-performing teams:

Metric	Measures	Better means
Deployment frequency	How often you release to production	More often
Lead time for changes	Commit to running in production	Shorter
Change failure rate	% of releases causing a problem	Lower
Time to restore service	How fast you recover from failure	Faster

KEY IDEA

Speed and stability **rise together**. Small, frequent, automated releases are *safer* than rare big ones.

Chapter recap

- ☐ CI builds and tests every commit; CD makes releases routine
- ☐ Pipelines fail fast and run the same checks for everyone
- ☐ A GitHub Actions workflow = events, jobs and steps in YAML
- ☐ Build an image once, promote it through environments
- ☐ Rolling, blue/green and canary limit release risk
- ☐ IaC (Terraform) makes infrastructure reviewable
- ☐ Secrets live in managers, never in code
- ☐ Monitor, set SLOs and measure with DORA

⚒ Hands-on lab: Your first pipeline

1. Put the Flask app and a `test_app.py` in a GitHub repository.
2. Add `.github/workflows/ci.yml` from this chapter and push.
3. Break a test on purpose in a pull request and watch the check fail.
4. Fix it, merge, and add the Docker build job.
5. Turn on branch protection so `main` needs a passing check.

Deployment Architecture: The Big Picture

CHAPTER 6

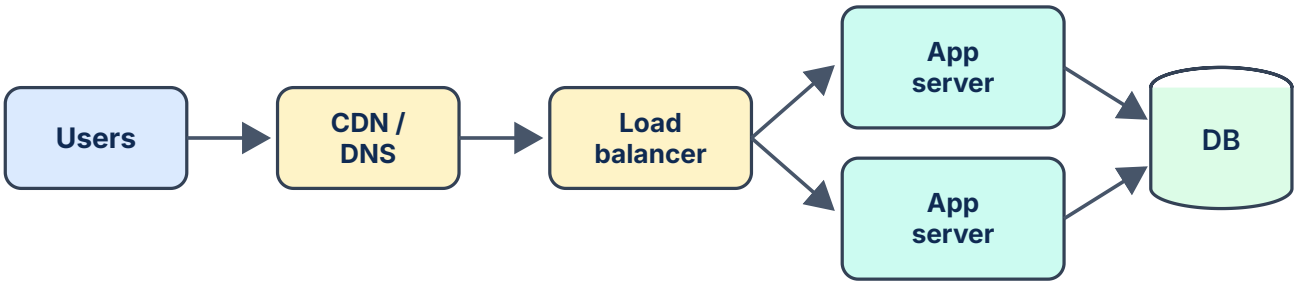
06 Deployment Architecture and Mini-Projects

Put every earlier idea to work

WHAT YOU WILL LEARN

- Read and draw cloud architecture diagrams
- Compare monolith and microservices; add caching and queues
- Apply the Well-Architected pillars
- Build four portfolio-ready mini-projects

Architecture is how the parts of a system are arranged and connected. A good deployment architecture answers four questions: *How do users reach it? Where does the code run? Where does the data live? What happens when something fails?*



A typical web architecture

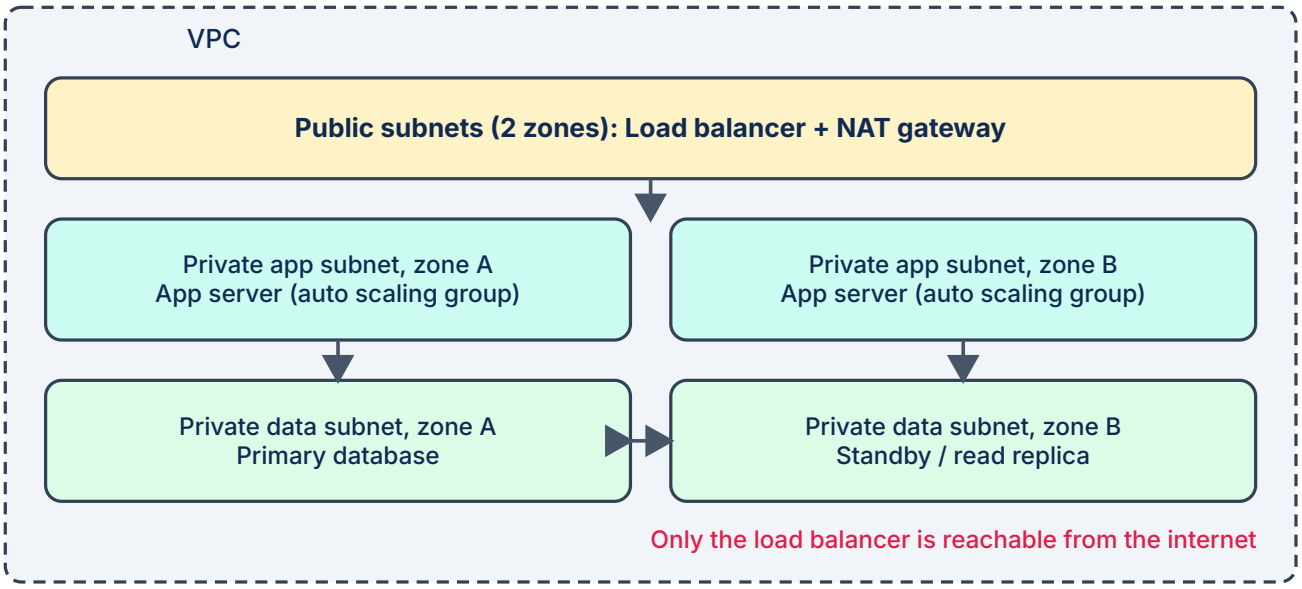
Layer	Job	Cloud examples
Edge	DNS, CDN, firewall	Route 53 / Azure DNS / Cloud DNS, CloudFront, WAF
Entry	Load balancing, TLS	ALB, Application Gateway, Cloud Load Balancing
Compute	Run application code	VMs, containers, serverless
Data	Databases, cache, files	RDS / Azure SQL / Cloud SQL, Redis, object storage

PRO TIP

When designing, **draw first**. If you cannot draw how a request travels and what fails if one box dies, you do not understand it yet.

The Three-Tier Architecture on the Cloud

Most web applications split into **presentation**, **application** and **data** tiers. Each tier lives in its own subnet with its own firewall rules.



Two zones, three tiers, layered security

Tier	Security group allows
Load balancer	443 from the internet
App servers	App port **only from the load balancer**
Database	DB port **only from the app servers**

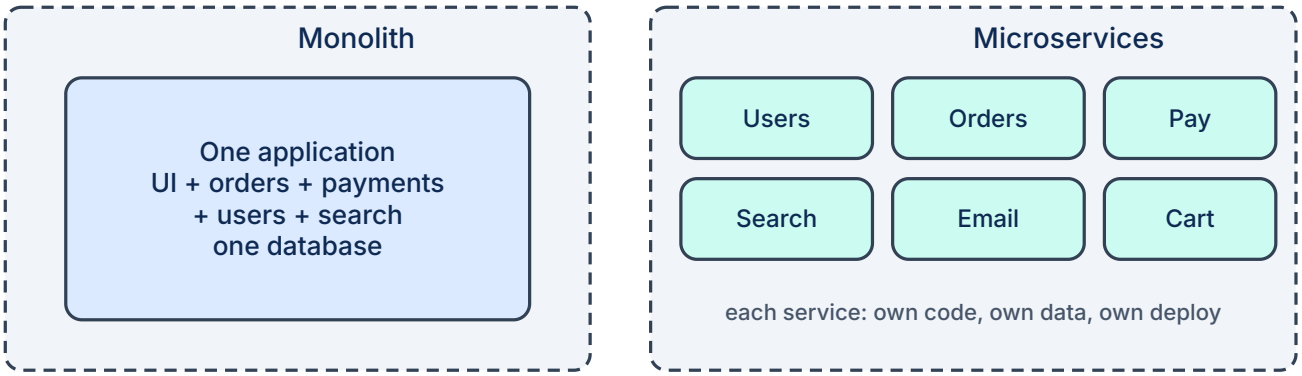
KEY IDEA

Each tier trusts only the tier in front of it. If an attacker breaks into one layer they cannot jump straight to the data.

NOTE

A NAT gateway lets private servers fetch updates without being reachable. It bills by the hour, so remember it during cleanup.

Monolith vs Microservices



One deployable unit vs many small ones

	Monolith	Microservices
Start-up speed	Fast, simple	Slower: more setup
Deploy	All or nothing	Each service alone
Scaling	Scale the whole app	Scale only the busy service
Failure	One bug can stop everything	Failures can be contained
Complexity	In the code	In the network, tracing, versions
Team size	Small teams	Many teams working in parallel

KEY IDEA

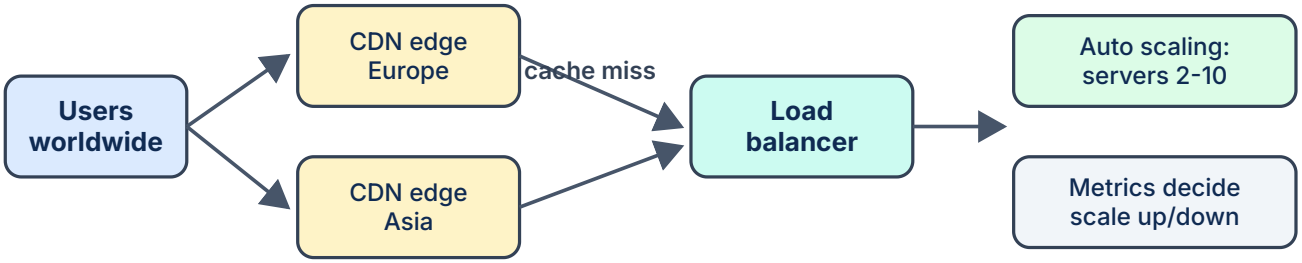
Start with a well-structured monolith and split services when real pain appears (team size, scaling, release speed). Many famous systems did exactly this.

WATCH OUT

Microservices without CI/CD, monitoring and containers become a distributed mess. This book's earlier chapters are the prerequisites.

Load Balancing, Auto Scaling and CDN

Three components make a site both fast and resilient.



The CDN answers most requests; the rest reach scaled servers

Component	Problem it solves	How
CDN	Distance and load	Caches static files (images, JS, CSS) at edge locations near users
Load balancer	One server cannot carry all traffic	Spreads requests, runs health checks, ends TLS
Auto scaling	Traffic changes	Adds or removes servers from metrics or schedules

Health checks

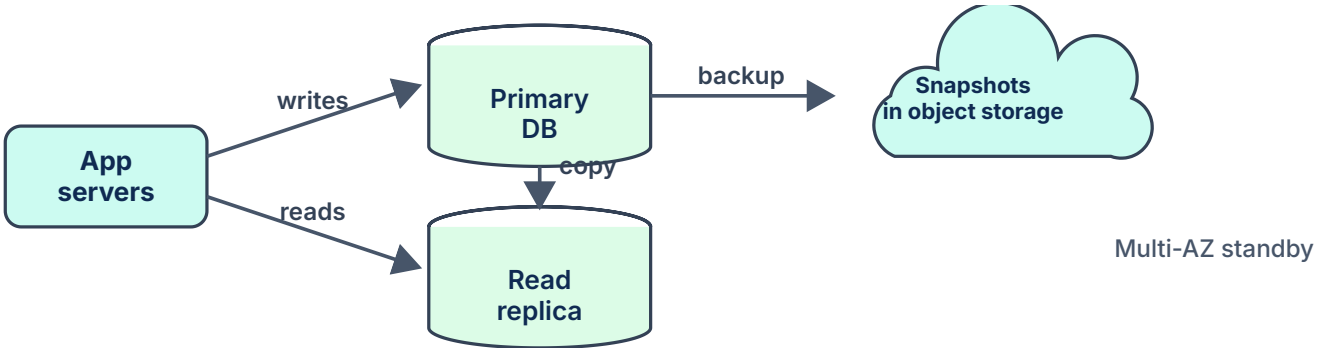
```
health check rule
GET /health  →  200 {"status": "ok"}

Load balancer: check every 10 s
Unhealthy after 3 failures → stop sending traffic
Healthy after 2 successes  → resume
```

PRO TIP

This is why the Flask app in Chapter 4 had a `/health` route. Make it check what truly matters (can I reach the database?), but keep it fast.

Databases, Replicas, Backups and Caching



Writes to the primary, reads from replicas

Technique	Purpose
Read replica	Spread read traffic; reporting
Multi-AZ standby	Automatic failover if the primary's zone fails
Automated backups + PITR	Restore to a moment in time (RPO)
Cache (Redis)	Answer repeated reads from memory in microseconds

Cache-aside pattern

cache_aside.py

```
cache = {}
db_reads = 0

def slow_db_lookup(user_id):
    global db_reads
    db_reads += 1
    return {"id": user_id, "name": f"user{user_id}"}

def get_user(user_id):
    if user_id in cache:            # 1. try the cache
        return cache[user_id]
    row = slow_db_lookup(user_id)   # 2. miss: ask the database
    cache[user_id] = row            # 3. remember it
    return row
```

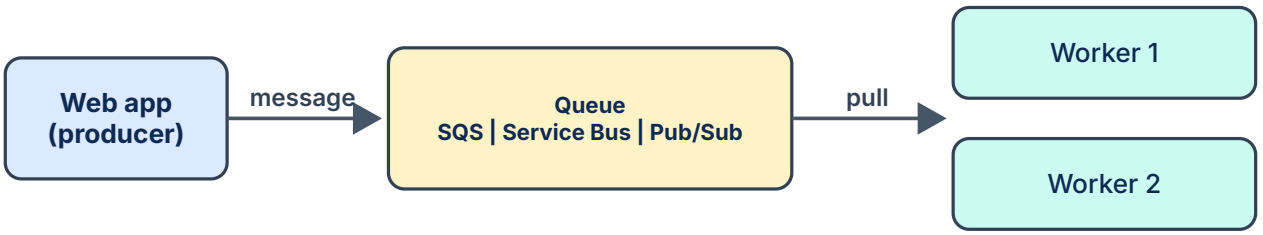
```
requests: 5, database reads: 0
```

WATCH OUT

Cached data can be **stale**. Set an expiry (TTL) and decide which data may be slightly old.

Event-Driven Design and Message Queues

When one service calls another directly and waits, a slow service slows everyone. A **message queue** decouples them: the sender drops a message and moves on; workers process it when ready.



The queue absorbs bursts of work

queue_demo.py

```
from collections import deque

queue = deque()
def place_order(order_id):
    queue.append({"order": order_id})
    return "accepted"          # the web request returns immediately

def worker():
    done = []
    while queue:
        msg = queue.popleft()
        done.append(f"emailed receipt for order {msg['order']}")
    return done
```

```
['accepted', 'accepted', 'accepted']
['emailed receipt for order 101', 'emailed receipt for order 102', 'emailed receipt
for order 103']
```

Concept	Meaning
Queue	One message is handled by one worker
Topic (pub/sub)	One message is copied to many subscribers
Dead-letter queue	Where messages go after repeated failure, for inspection
Idempotent	Processing the same message twice gives the same result

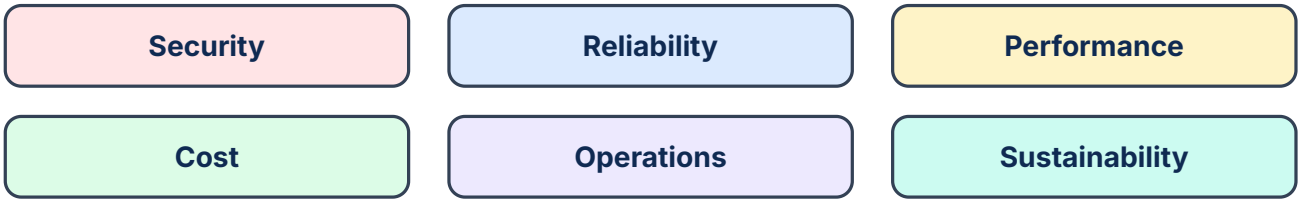
KEY IDEA

Queues deliver messages **at least once**, so duplicates happen. Design workers to be idempotent.

The Well-Architected Pillars

AWS, Azure and Google Cloud each publish a **Well-Architected Framework** that is a checklist for good design. The pillars overlap strongly:

Pillar	Ask yourself	Example practice
Operational excellence	Can we run, observe and improve it?	CI/CD, runbooks, dashboards
Security	Who can access what, is data protected?	Least privilege, encryption, MFA
Reliability	Does it recover from failure?	Multi-AZ, backups, health checks
Performance efficiency	Right resources for the load?	Caching, right-sizing, CDN
Cost optimisation	Are we paying only for value?	Budgets, tags, auto scaling
Sustainability	Is energy use minimised?	Efficient instance types, scale to zero



Good design is a balance: improving one pillar often costs

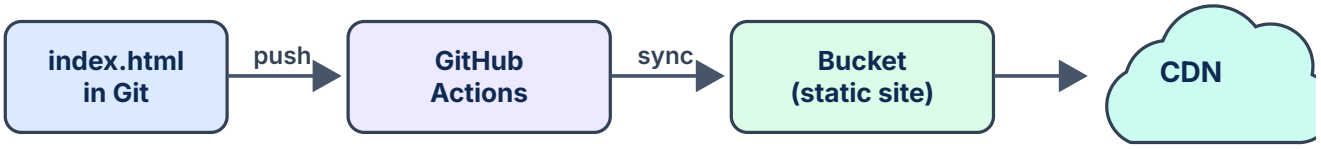
Six pillars, always traded off

PRO TIP

In interviews, structure any design answer around these pillars: *"For reliability I would... for security I would..."*. It sounds organised because it is.

Mini-Project 1: Static Website on Object Storage

Goal: host your portfolio page from a bucket and deploy it automatically on every push.



Git push updates the live site

1. Create a bucket (S3 / Blob / Cloud Storage) with static website hosting and write `index.html`.
2. Allow public reads **only** for the site files (or put a CDN in front).
3. Add the workflow below; use OIDC or a least-privilege deploy role.
4. Open the site URL, change a word, push, refresh.

`.github/workflows/deploy.yml (AWS version)`

```yaml
name: Deploy site
on:
  push:
    branches: [main]
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/site-deployer
          aws-region: ap-south-1
      - run: aws s3 sync ./site s3://my-portfolio-2026 --delete
```

WATCH OUT

`--delete` removes remote files that no longer exist locally. Test with a small bucket first.

Mini-Project 2: A Flask + Redis App (Code)

Goal: a hit-counter web app that stores its count in Redis, so it keeps working correctly when you run several copies. First the code, tested here with a fake Redis:

app.py

```
import os
from flask import Flask, jsonify

def create_app(store=None):
    app = Flask(__name__)
    if store is None:
        import redis
        store = redis.Redis(host=os.getenv("REDIS_HOST", "localhost"),
                            port=6379, decode_responses=True)

    @app.get("/")
    def home():
        return jsonify(hits=store.incr("hits"))

    @app.get("/health")
    def health():
        return jsonify(status="ok")

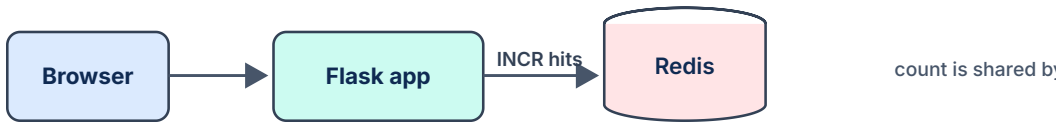
    return app

app = create_app() if os.getenv("REDIS_HOST") else None
```

```
[1, 2, 3]
{'status': 'ok'}
```

requirements.txt

```
flask==3.1.3
gunicorn==23.0.0
redis==5.2.1
```



State lives in Redis, not in the app

KEY IDEA

`create_app()` takes its storage as a parameter, so tests can inject a fake Redis (dependency injection).

Mini-Project 2: Containerise and Compose

Dockerfile

```
FROM python:3.12-slim
ENV PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
RUN useradd --create-home appuser
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

compose.yaml

```
services:
  web:
    build: .
    ports:
      - "8080:5000"
    environment:
      REDIS_HOST: cache
    depends_on:
      cache:
        condition: service_healthy
  cache:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
```

run it (illustrative)

```
$ docker compose up -d --build
$ curl localhost:8080/
$ curl localhost:8080/
$ docker compose up -d --scale web=3   # needs a load balancer to use the 3
copies
$ docker compose down
```

1. Run `curl localhost:8080` repeatedly; the count increases.
2. Run `docker compose restart web`; the count is **kept** because Redis holds it.

Mini-Project 3: CI/CD Pipeline to the Cloud

Goal: every merge to `main` tests, builds and deploys the container to **Cloud Run**. (On AWS use ECS/App Runner; on Azure use Container Apps. The pipeline shape is identical.)



Test, package, deploy, verify

.github/workflows/deploy.yml

```yaml
name: Deploy
on:
  push:
    branches: [main]
permissions:
  id-token: write
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt pytest && pytest -q
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: ${{ secrets.WIF_PROVIDER }}
          service_account: ${{ secrets.DEPLOY_SA }}
      - id: deploy
        uses: google-github-actions/deploy-cloudrun@v2
        with:
          service: shop-api
          region: asia-south1
          source: .
      - run: curl --fail "${{ steps.deploy.outputs.url }}/health"
```

KEY IDEA

Credentials come from **workload identity federation (OIDC)**: no JSON key is stored anywhere. The three values are stored as repository secrets.

Mini-Project 4: Terraform a Server with Monitoring

Goal: create a web server and a CPU alarm using only code, then destroy it all with one command.

main.tf

```
variable "ami_id" {
  description = "AMI for Amazon Linux in your region"
  type        = string
}

resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"
  tags          = { Name = "devops-lab-web" }
}

resource "aws_cloudwatch_metric_alarm" "cpu_high" {
  alarm_name          = "web-cpu-high"
  namespace           = "AWS/EC2"
  metric_name         = "CPUUtilization"
  dimensions          = { InstanceId = aws_instance.web.id }
  statistic           = "Average"
  period              = 300
  evaluation_periods  = 2
  threshold           = 80
  comparison_operator = "GreaterThanThreshold"
}
```

workflow (illustrative)

```
$ terraform init
$ terraform plan -var ami_id=ami-0123456789abcdef0
$ terraform apply -var ami_id=ami-0123456789abcdef0
$ terraform destroy -var ami_id=ami-0123456789abcdef0
```

- ☐ Commit the `.tf` files, **not** `terraform.tfstate` or `.terraform/`
- ☐ Repeat the server with an Azure VM or a Google Compute Engine instance

WATCH OUT

Always finish with `terraform destroy` for lab environments. Forgotten servers are the classic way to pay for nothing.

Career Roadmap and Portfolio



A realistic learning path

Role	Focus	Add next
Cloud engineer	Build and run cloud infrastructure	Networking, IAM, Terraform
DevOps engineer	Pipelines, automation, releases	Kubernetes, scripting, observability
Site reliability engineer	Reliability, SLOs, incident response	Systems design, on-call practice
Platform engineer	Internal tools for developers	Kubernetes, IaC, developer experience
DevSecOps	Security in the pipeline	Scanning, secrets, compliance

A portfolio that gets interviews

- ☐ Four GitHub repos from the mini-projects, each with a clear README
- ☐ An architecture diagram and a short *"what I learned"* section per project
- ☐ A live URL or screenshot of the running app
- ☐ Meaningful commit history and a green CI badge
- ☐ One cloud certification to match your hands-on work

A 12-week plan

Weeks	Focus
1-2	Linux, Git, shell scripting daily
3-4	Docker, Compose; containerise two apps
5-6	CI/CD with GitHub Actions
7-9	One cloud: VPC, VMs, storage, IAM, Terraform
10-12	Mini-projects, certification, mock interviews

DevOps Interview Questions

Short, honest answers beat long memorised ones. Practise saying these out loud.

Question	Strong short answer
What is the difference between a container and a VM?	A VM virtualises hardware with its own OS; a container isolates a process and shares the host kernel, so it is lighter and faster.
Explain CI vs CD.	CI automatically builds and tests every merge; CD keeps the build releasable (delivery) or deploys it automatically (deployment).
What is a Docker layer, and why does order matter?	Each instruction creates a cached layer. Put rarely changing steps first so rebuilds reuse the cache.
How would you give an app access to a cloud bucket safely?	Attach a role / managed identity / service account with least privilege; never embed keys.
Blue/green vs canary?	Blue/green switches all traffic between two full environments; canary shifts a small share first and watches metrics.
What does `chmod 755` do?	Owner can read, write, execute; group and others can read and execute.
How do you roll back a bad release?	Redeploy the previous image tag (or flip blue/green); reversible migrations and feature flags help.
What is Infrastructure as Code?	Defining infrastructure in version-controlled files so it is reviewable, repeatable and automatable.
Where do secrets go?	A secret manager or CI secret store, injected at runtime; never in Git or images.
Pod vs container in Kubernetes?	A pod is the smallest deployable unit and wraps one or more containers sharing network and storage.

PRO TIP

When you do not know something, say how you would **find out**: logs, docs, a small experiment. Interviewers hire people who can learn.

Glossary, References and Next Steps

- **AZ**: isolated data-centre zone in a region
- **CI/CD**: automated build, test and release
- **CIDR**: notation for IP ranges (10.0.0.0/16)
- **Container**: isolated packaged process
- **IaC**: infrastructure defined in code
- **IAM**: identity and access management
- **Image**: read-only container template
- **Pipeline**: automated stages from commit to deploy
- **RPO / RTO**: tolerable data loss / downtime
- **SLO**: reliability target for a service
- **Subnet**: slice of a VPC network
- **VPC**: private virtual network
- **Volume**: storage that outlives a container

References and further reading

- NIST SP 800-145, *The NIST Definition of Cloud Computing*
- Official docs: docs.docker.com, docs.github.com/actions, developer.hashicorp.com/terraform
- AWS, Azure and Google Cloud documentation and Well-Architected frameworks
- Google, *Site Reliability Engineering* (free at sre.google/books)
- DORA research programme, *Accelerate State of DevOps* reports

> **NOTE**
> Cloud services, versions and free-tier rules change often. Treat the official documentation as the source of truth and re-check commands before running them on a real account.

Keep learning with PrepPath E-books Store

Explore more e-books on programming, interview preparation and technology.
Visit our website and shop now:

preppath-ebooks-store.vercel.app